Compiler: Firstly, it checks the syntax, once error free then it converts to low-level code

Lexical Analysis: Lexims are categorized under tokens

Tokens:
Keywords (set of predefined words)
Identifiers
Operators
Special Symbols
Constants

Each lexim in code is categorized under tokens
After categorization, we go for code generation

r = a + b * c - d

$t_1 = b * c;$
$t_2 = a + t_1;$
$t_3 = t_2 - d;$
$r = t_3$

code Optimization: reducing the instructions wherever possible

code generation: Not 0's & 1's compiler generates Assembly code, where assembler converts to low-level language

Phases:
→ Lexical Analysis
→ Syntax Analysis
→ Semantic Analysis
→ Intermediate code generation
→ Code Optimization
→ Code generation.

r = a + b * c - d

Errors:-
"ab is not declared"

formal Languages for Automata Theory:

→ formal grammar ways:

1. Regular G → (Lexical Analysis) Regular Expression: email_id patterns finite Automata phases of compiler are expressed under RG
2. Context free G → (Syntax Analysis) Push Down Automata (describes hierarchical structure of the language)
3. Context Sensitive G → Linear Bounded Automata
4. Unrestricted G → Turing Machine

"Now tokens are verified / should be in Grammar"

① Not sufficient for Syntax Analysis To find the longest grammar we need Automata

② context free used in Syntax Analysis
comparision among open parantheris = close parantheris

③ context sensitive in semantic Analysis
checking the declaration & usage of variable (within scope)

④ Unrestricted not related to compiler Analysis
"suitable for Algorithms"

→ To check whether solution exist / not (Yes/No type)

Turing Machine ∴ Mach can understand these are formal language

Automata: If it is as per the grammar, It gives Yes/No

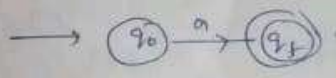
→ It is a machine (Yes/No type)

→ If we have grammar then we need to build suitable Automata

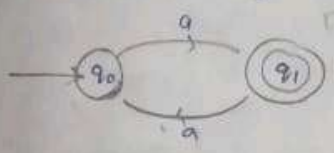
strings over {a} having even length

{ Σ, S, q_0, q_f } G: S → aS|ε

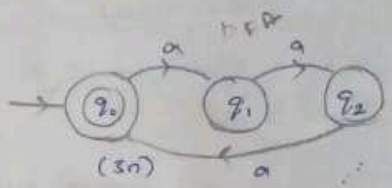
* Set of all strings with a common condition is language (so we can have ∞ langs)



L = {a}

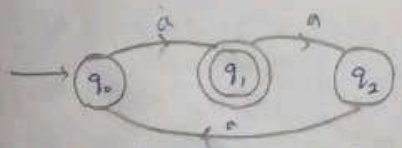


L = {a, aaa, aaaaa, ...}

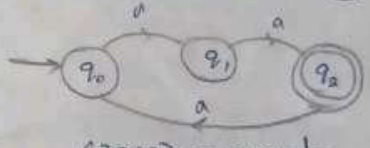


Strings over A who are divisibu by 3

L = {aaa, aaaaaa, ...}



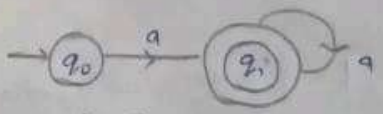
(3n+1) or 1 mod 3



(3n+2) or 2 mod 3



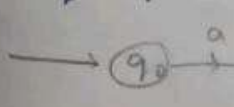
$L = \{a^n \mid n \geq 0\}$



$L = \{a^n \mid n \geq 1\}$

⇒ Construct the Automata which accepts strings over {a,b} that starts with a

L = {a, aa, ab, aaa, aba, aab, abb, ...}

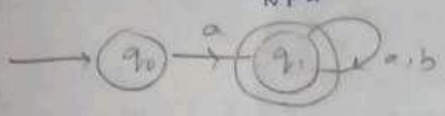


Regular Expression: $a(a+b)^*$

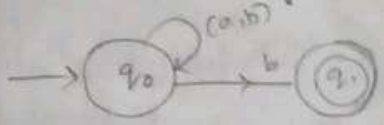
2.) construct the finite automata that accepts all the strings over {a,b} that starts with 'a'

(Automata)
It has to accept all the strings that are there in the language.

NFA



3.) construct the finite automata that accepts all the strings over {a,b} that end with 'b'



Deterministic
for every i/p symbol no dilemma for computing the string.

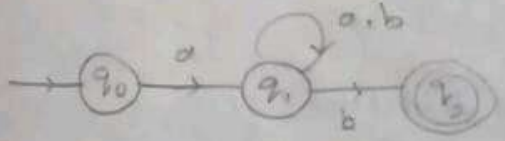
$L = \{b, ab, bb$

non-Determ

Non-Deterministic

4.) construct the finite automata that accepts all the strings over {a,b} that start with 'a' & end with 'b'

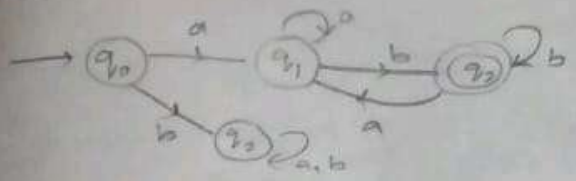
If they exist '0' or more than '1' transition



Finite Automata

Deterministic finite Automata

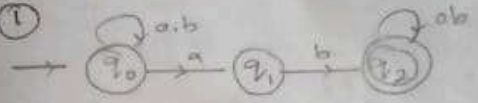
Non-Deterministic finite Automata



deadstate
q_3 itself

construct finite automata that accepts all strings over $\{a, b\}$.

i) That has substring 'ab' $(a|b)^* ab (a|b)^*$



(NFA)

q_0 has more than 1 transition for a

Transition diagram → one representation (diagramatic)
→ Tabular representation

② final state → *.

present \ I/P	a	b
$\rightarrow q_0$	$q_0 q_1$	q_0
q_1	-	q_2
$*q_2$	q_2	q_2

③ ← Descriptive representation.

$Q : \{q_0, q_1, q_2\}$

$\Sigma : \{a, b\}$

$\delta : \delta(q_0, a) = \{q_0, q_1\}$

$\delta(q_0, b) = q_0$

$\delta(q_1, b) = q_2$

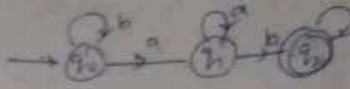
$\delta(q_2, a) = q_2$

$\delta(q_2, b) = q_2$

Initial $I_S : q_0$

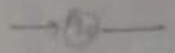
final $F : \{q_2\}$

{ab, aab, bab, aba, aba, abb ... }



in terms of acceptance

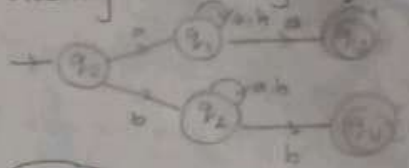
$a^* a\, b$



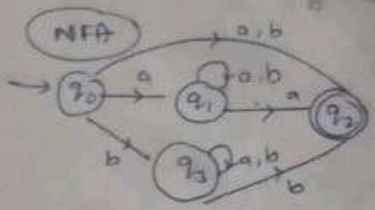
q_0

construct finite automata

Starting & ending symbols are same.

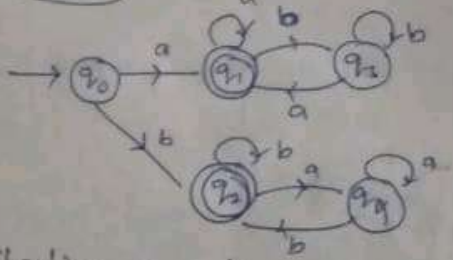


NFA

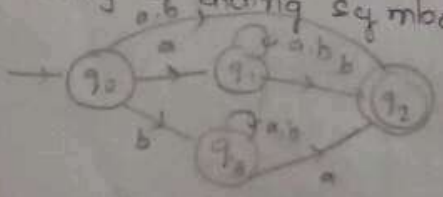


NFA

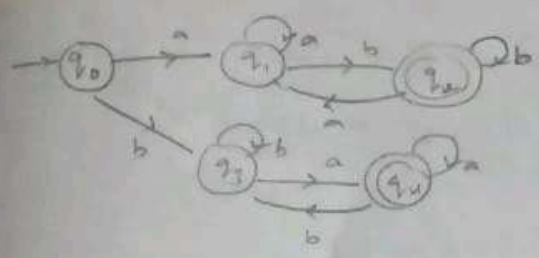
DFA



Starting & ending symbols are different.



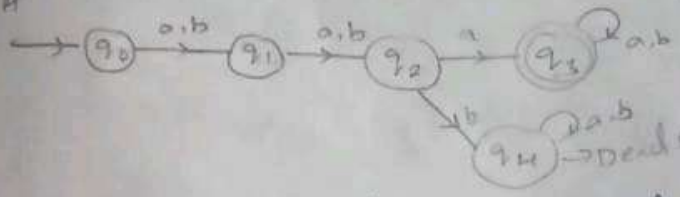
NFA



check for multiple 'b' is it satisfy?

Third symbol from starting is a.

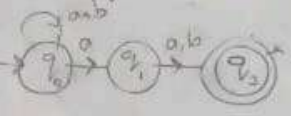
DFA



Transitions has to be done till the last symbol.

① last but one symbol is 'a' (from right → 2nd symbol)

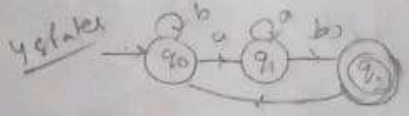
The decision changes



NFA

aa

DFA

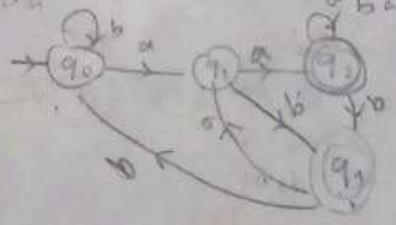


violates

aa, baba, aaaba

aab?

abaa?



aa, ab, aaa, aaba, baa

$(q_1, b) = a$

(q_0, b)

$(q_3, a) \rightarrow aa, ab$

(q_3, a) ab

(2) → length of the string is exactly '3'.

(3) → length of the string is atleast '3'

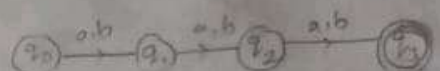
(4) → " " " " atmost '3'

(5) → no. of a's divisible by 2.

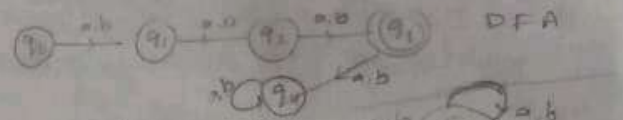
(6) → no. of a's & b's also divisible by 2.

ab ✓ True aa, bb

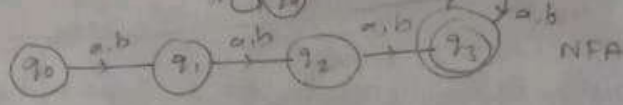
(3) NFA



DFA

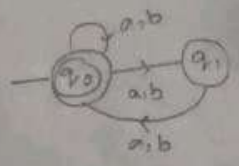


(3) NFA



DFA

(4)



1 2 3

ε, a, aa, aaa,

0, a, b, aa, bb, ab, ba,

aaa, aab
bba, bbb
aba, abb
baa, bab

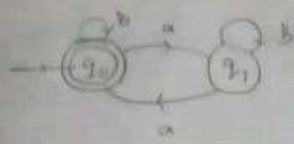
both NFA & DFA because min no. of {a,b}* n=2

States → n+1

= 3

abab not accepted ✗

(5)

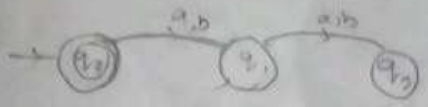


q_0, q_1, q_2

ϵ, aa, aab, aba

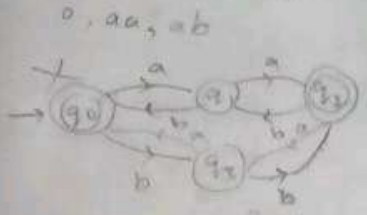
NIRZDFA

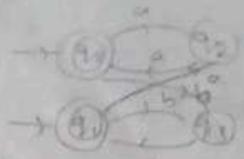
(6)



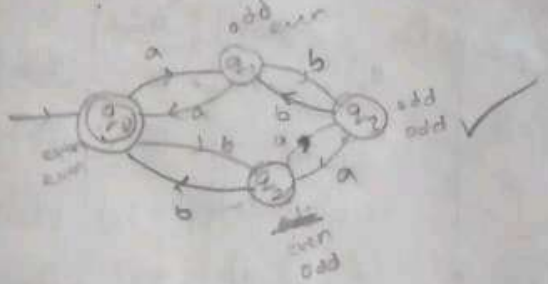
$\{\epsilon, aa, bb, aabb\}$

abab, abba, baab

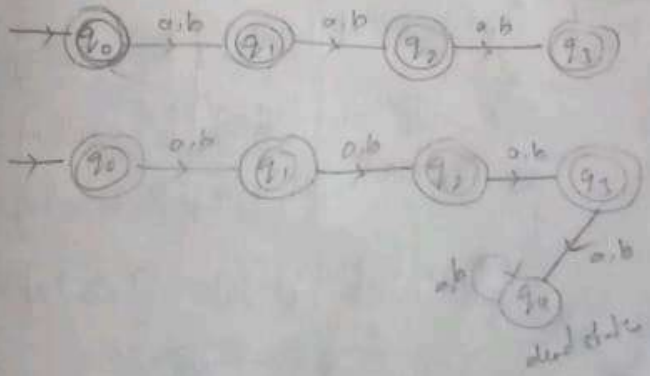




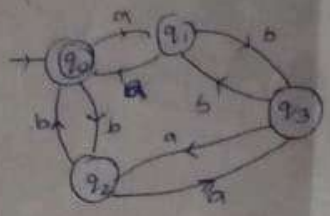
q_0 0 0
q_0 1 2
q_2 0 1
q_1 1 0



(4)



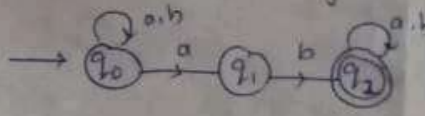
③ no of a's & b's divisible by 2



NFA = DFA

	a	b
q_0	q_1	q_2
q_1	q_0	q_3
q_2	q_3	q_0
q_3	q_2	q_1

④ That has substring 'ab' NFA → no dead



NFA

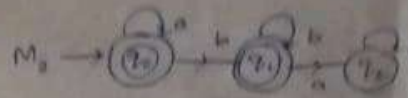
NFA

	a	b
q_0	$q_0 q_1$	q_0
q_1	–	q_2
*q_2	q_2	q_2

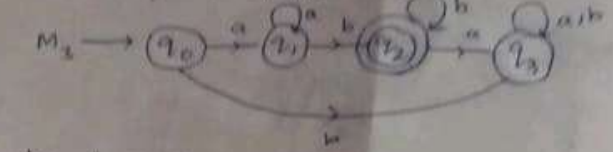
DFA

	a	b
q_0	$[q_0 q_1]$	q_0
$[q_0 q_1]$	$[q_0 q_1]$	$[q_0 q_2]$
*$[q_0 q_2]$	$[q_0 q_1 q_2]$	$[q_0 q_2]$
*$[q_0 q_1 q_2]$	$[q_0 q_1 q_2]$	$[q_0 q_2]$

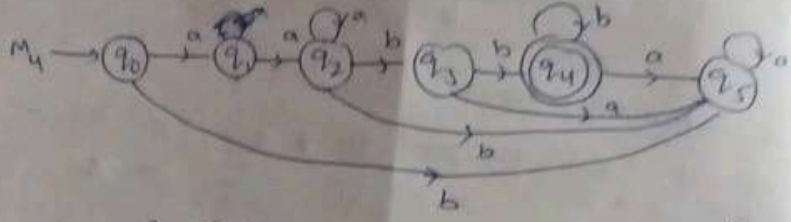
$L_2 = \{a^m b^n \mid m, n \geq 0\}$ → compared with parenthesis as parenthesis



$L_3 = \{a^m b^n \mid m, n \geq 1\}$



$L_4 = \{a^m b^n \mid m, n \geq 1\}$

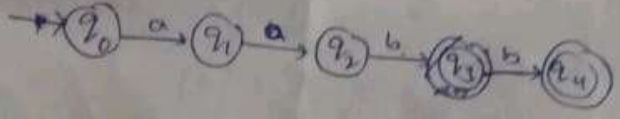


$L_5 = \{a^m b^n c^p \mid m, n, p \geq 1\}$ minimal no. of states = 3



$L_6 = \{a^m b^m \mid m = 2\}$ $a^2 b^2 = aa\,bb$

$a^4 b^4$ $2*2$



$L_8 = \{a^m b^m \mid m \geq 0\}$ length of string ≥ 1

↓

true the m is never-ending, so no. of state are never-ending so it is not regular.

upper limit → finite string → finite Automata → Regular

Minimization of DFA — MYHILL NERODE THEOREM

	a	b
A	B	E
B	C	D
* C	H	I
* D	I	H
E	F	G
* F	H	I
* G	H	I
H	H	H
I	I	I

	A	B	C	D	E	F	G	H
B	x							
C	x	x						
D	x	x	o					
E	x	o	x	x				
F	x	x	o	o	x			
G	x	x	o	o	x	o		
H	x	x	x	x	x	x	x	
I	x	x	x	x	x	x	x	o

states with similar behaviour will be grouped

step-1: First distinguishability will be found b/w final & non-final state

step-2: If there are 2 states

$\delta(p,a) = r$

$\delta(q,a) = s$

If r & $s \rightarrow (r,s)$ pair is distinguishable the (p,q) pair also distinguishable

After applying 2 steps repetitively the remaining states are marked as equivalent

① {C, D, F, G}
{A, B, E, H, I}

$n(n-1)/2$

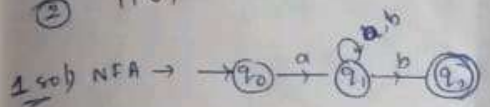
② (A, B) × based on (B, C)
(A, E) × based on a (B, F)
(B, H) × based on a (C, H)
(B, I) based on (G, I)
(A, H)
( )

	a	b
A B	AB	F
C D	GH	AB
E	AB	GH
* F	GH	CD
* G H	AB	CD

① starting with a & ending with b

② 1101

1 sol) NFA →



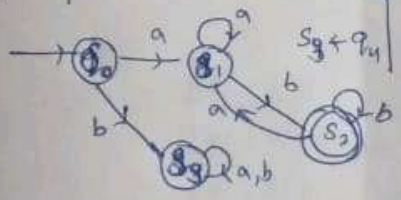
NFA table

	a	b
q_0	q_1	-
q_1	q_1	$q_1 q_2$
* q_2	-	-

DFA table

	a	b
$S_0 \leftarrow q_0$	q_1	q_4 (DS)
$S_1 \leftarrow q_1$	q_1	$[q_1 q_2]$ → S_2
~~$S_2 \leftarrow$ * q_2~~	~~q_4~~ (DS)	~~q_4~~ (DS)
$S_3 \leftarrow q_4$	q_4	q_4

$[q_1 q_2]$ → q_1 $[q_1 q_2]$



S_0 S_1 S_3
S_1
S_2
S_3

	a	b
S_0	S_1	S_3
S_1	S_1	* S_2
* S_2	S_1	S_2
S_3	S_3	S_3

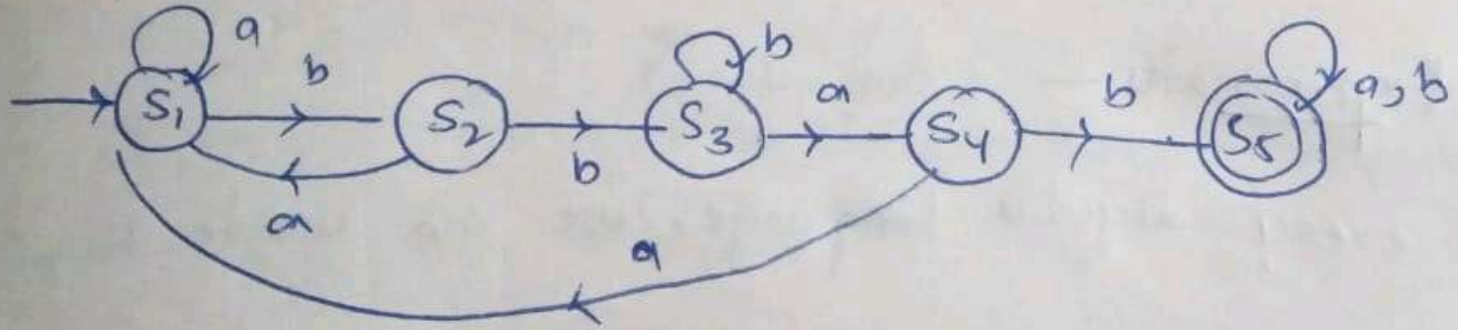
S_1	X		
S_2	X	X	
S_3	X	X	X
	S_0	S_1	S_2

① $\{S_0, S_1, S_3\}$ $\{S_2\}$

$\{S_0, S_1\}$ → on b → $\{S_3, S_2\}$

$\{S_0, S_2\}$ → (q_1, S_3)

$\{q_1, S_3\}$ → on b → $\{S_2, S_3\}$


a
b
b
a
b
a
b
a, b
S1
S2
S3
S4
S5
b
a

② All the strings excluding Epsilon

$(a+b)^+$ ⟶ positive closure (some language without epsilon)

(un = 1 to ∞)

Kleene closure (un = 0 to ∞)

③ All strings starting with a.

$a(a+b)^*$ [a followed by any string]

④ All strings ending with b

$(a+b)^*b$

⑤ strings either starting with a or ending with b.

$(a(a+b)^* + (a+b)^*b)$

↓ either/or

⑥ Startings with 'a' & ending with b.

$a(a+b)^*b$

⑦ strings with 'ab' substring.

$(a+b)^*ab(a+b)^*$

⑧ 'ab' prefix $ab(a+b)^*$

⑨ 'ab' suffix $(a+b)^*ab$

⑩ Where strings whose length is divisible by 2.

$[(a+b)^2]^*$

⑪ Strings which contains exactly two a's

$b^*ab^*ab^*$

⑫ Atleast 2 a's

$(a+b)^*a(a+b)^*a(a+b)^*$

⑬ utmost 2 a's

$b^*(a+\epsilon)b^*(a+\epsilon)b^*$

- Reg Ex satisfy A.P over union.

$r_1 + (r_2 + r_3) = (r_1 + r_2) + r_3$

- Reg Ex satisfy A.P over concatenation

$r_1 \cdot (r_2 r_3) = (r_1 r_2) r_3$

* Distributive Property:-

Distribul

- Distribution of union over concatenation is not satisfied.

$r_1 + (r_2 r_3) \neq (r_1 + r_2)(r_1 + r_3)$

- Distribution of concatenation over union is satisfied

$r_1 (r_2 + r_3) = r_1 r_2 + r_1 r_3$

$(r_1 + r_2) r_3 = r_1 r_3 + r_2 r_3$

* Identity Property / Element:-

It is constant expression.

$r \text{ op } X = r$

** ϕ is identity over union operation

$r + \phi = r$

* ε is identity over concatenation

$r \cdot \varepsilon = r$

* Annihilator Property:-

r op $X = X$ returns

- There is Annihilator over union
- $r \cdot \phi = \phi$ ϕ is Annihilator over concatenation

Conversion Finite Automata to Regular Expression:-

2 methods

1. Arden's Method
2. State Elimination method.

Arden's Lemma:- R → state

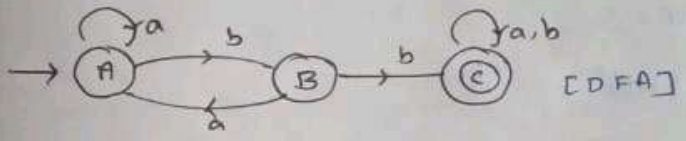
- If P & Q are regular expressions then
  1. The Eqn $R = Q + R \cdot P$ has a unique solution,
  2. if 'P' doesn't contain 'ε' and the solution is $R = Q \cdot P^*$
  3. The Eqn $R = Q + R \cdot P$ has infinitely many solutions, if 'P' contains 'ε'

Algorithm:-

step-1: write characteristic equations (CE) for every state using state transition diagram.

step-2: Solve the C.Eqns for getting the eqn of final state solution.

step-3: Apply the Arden's lemma wherever it is applicable for removal of states in the equation and final states expression is the required regular expression



[DFA]

$A = \epsilon + Aa + Ba$ — ①

$B = Ab$ — ②

$C = Bb + C(a+b)$ — ③

Substituting ② in ①

$A = \epsilon + Aa + Aba$

$A = \epsilon + A(a + ba)$ (in $R = Q + RP \rightarrow R = QP^*$)

Applying Lemma

$A = \epsilon(a+ba)^* = (a+ba)^*$

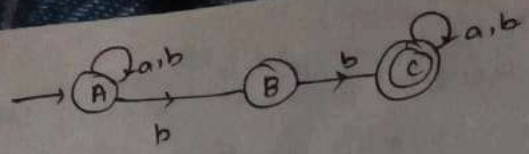
$R.\epsilon = (a+ba)^* bb(a+b)^*$

②: $B = (a+ba)^*(b)$

③: $C = \underset{R}{(a+ba)^*}\underset{Q}{b.b} + \underset{R}{C}\underset{P}{(a+b)}$

Apply Lemma.

$C = (a+ba)^* bb(a+b)^*$



A : $\epsilon + A(a+b)$ — (1)

B : Ab — (2)

C : $Bb + C(a+b)$ — (3).

~~Substitute (B) in (A)~~

$A = \epsilon + A(a+b)$

Apply Lemma.

$A = \epsilon(a+b)^* = (a+b)^*$

$B := (a+b)^* b$

C : $(a+b)^* b \cdot b + C(a+b)$

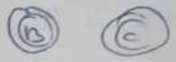
R.E = $(a+b)^* b\,b\,(a+b)^*$

Substring of length 'bb' 2'

* Do not forget 'ε' to add

* regular expression is alternative way of expressing regular language.

- we need final state soln only.
- when multiple states as final state then we need to find all final states solution & combine them



B + C.

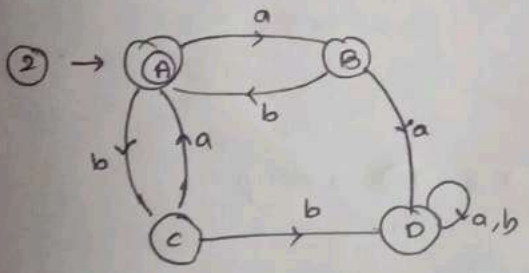
- Situation: $A = A_ + B_$ Apply Umm.

$R = Q + RP$ $\quad R = QP^*$ 'R' will be removed

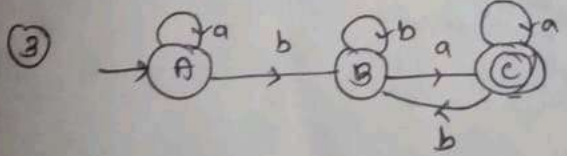
→ order changes then regular expression but its language is same.

Problems :-

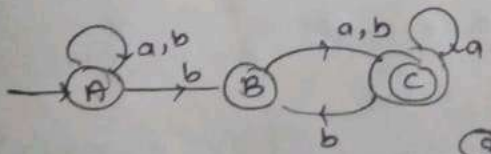
(2)



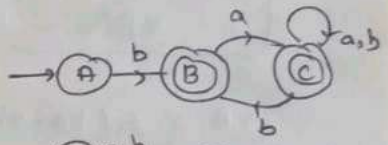
(3)



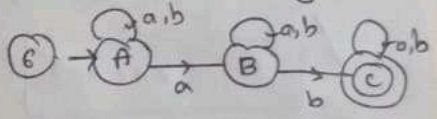
(4)



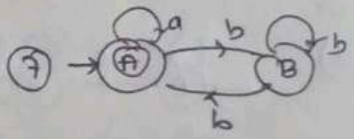
(5)



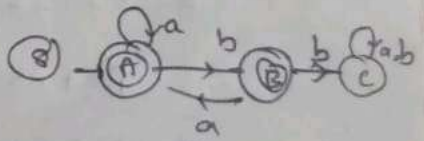
(6)



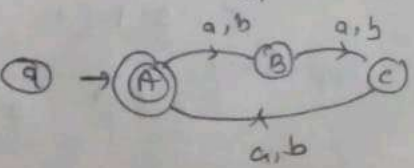
(7)



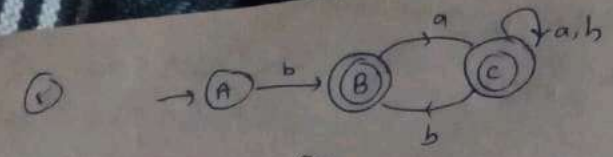
(8)



(9)



(5)



A : ε + ——— (1)

B : Ab + Cb ——— (2)

C : Ba + C(a+b) ——— (3)

In C Apply Lemma

→ C = Ba + C(a+b) [R = RP + Q]

R = QP*

C = Ba(a+b)*

B = ε·b + Cb

B = b + Ba(a+b)*b

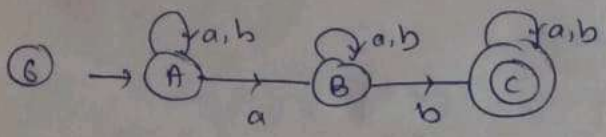
B = b[a(a+b)*b]*

C = b[a(a+b)*b]* a (a+b)*

Two final states → B + C ⇒ b[a(a+b)*b]* +

b[a(a+b)*b]*a(a+b)*

(6)



A : ε + A(a+b) ——— (1)

B : Aa + B(a+b) ——— (2)

C : C(a+b) + Bb ——— (3)

Apply Lemma on (1)

A = (a+b)*

B = (a+b)*a + B(a+b)

B = (a+b)*a(a+b)*

$B = a^*b + Bb + cb$

$B = Bb + a^*b + Qa^*b$

$B = Bb + a^*b(Q+1)$

$B = a^*b(Q+1)b^*$

$B = a^*b(Ba+1)b^*$

$C = Ba + Ca$

$C = Q + Ca$. [$\because Q = B$]

$C = Qa^*$

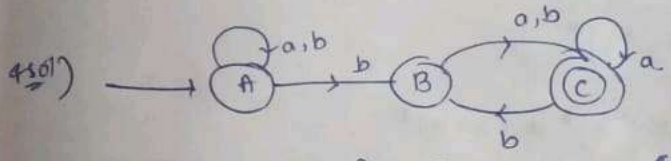
$B = Bb + a^*b + Ba a^*b$

$B = B(b + aa^*b) + a^*b$

$B = a^*b(b+aa^*b)^*$

$C = [a^*b(b+aa^*b)^*]a + Ca$

$C = [a^*b(b+aa^*b)^*]a \cdot a^*$



A: $\varepsilon + A(a+b)$ —①

B: $Ab + Cb$ —②

C: $B(a+b) + Ca$ —③

$C = B(a+b) + Ca$

$C = B(a+b)a^*$

from ① Apply Lemma.

A: $\varepsilon + A(a+b)$ [$\because R = RP + Q$]

$R = QP^*$

$A = \varepsilon(a+b)^*$ [$\because \varepsilon(a+b)^* \cdot \varepsilon = r^*$]

$A = (a+b)^*$

$B = (a+b)^*b + cb$

$B = (a+b)^*b + B(a+b)a^*b$

$B = (a+b)^*b[(a+b)a^*b]^*$

$C = (a+b)^*b[(a+b)a^*b]^*(a+b)a^*$

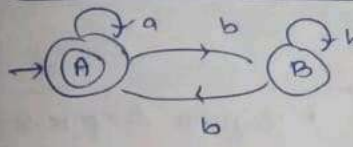
$(a+b)^*b(a+b)[b(a+b)+a]^*$

$C = C(a+b) + B(a+b)^* a(a+b)^* b$

$C = (a+b)^* a (a+b)^* b (a+b)^*$

Regex = $(a+b)^* a (a+b)^* b (a+b)^*$ ✓

(7)



$A = A\varepsilon + Aa + Bb$ — ①

$B = Ab + Bb$ — ②

Apply lemma on ②

$B = Abb^*$

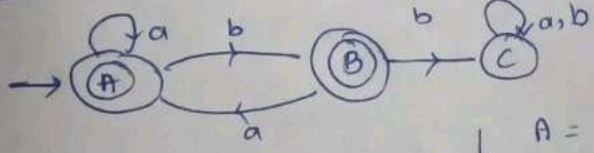
$A = \varepsilon + Aa + Abb^* b$

$A = \varepsilon + A(a + bb^* b)$

$A = \varepsilon (a + bb^* b)^*$

Regex = $[a+bb^* b]^*$ ✓

(8)



$A: \varepsilon + Aa + Ba$ — ①

$B: Ab$ — ②

$C: Bb + C(a+b)$ — ③

Sub ② in ①

$A = \varepsilon + Aa + Aba$

$A = \varepsilon + A(a + ba)$

$A = \varepsilon (a+ba)^*$

$A = (a+ba)^*$

$B = (a+ba)^* b$

$C = (a+ba)^* b \cdot b + C(a+b)$

$C = (a+ba)^* b \cdot b (a+b)^*$

Regex :- $A+B$

$(a+ba)^* + (a+ba)^* b$ ✓

Properties of Regular Languages:-

① closure
② Pumping Lemma.

closure Properties of R.L:-

- **Union Operator:-**

Let L_1, L_2 be two regular languages

$L_1 \cup L_2 = \{ x \mid x \in L_1 \text{ or } x \in L_2 \}$

- **Intersection:**

$L_1 \cap L_2 = \{ x \mid x \in L_1 \text{ and } x \in L_2 \}$

- **concatenation:**

$L_1 L_2 = \{ xy \mid x \in L_1 \text{ and } y \in L_2 \}$

- **Difference:**

$L_1 - L_2 = \{ x \mid x \in L_1 \text{ and } x \notin L_2 \}$

- **kleene closure:-**

$L^* = \{ (s_1 + s_2 + s_3 \cdots)^* \mid s_1, s_2, s_3 \in L \}$

$L = \{a, b, ab\}$

$L^* = \{ \in, a, b, ab, aa, ab, aab, \cdots \}$

$L^* = (s_1 + s_2 + s_3 + \cdots)^*$

* **Positive closure:-**

L^+

* **Symmetric Diff:-**

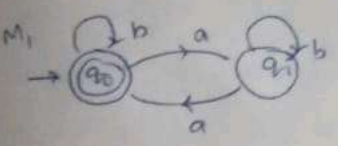
$L_1 \Delta L_2 = \{ x \mid x \in L_1 \cup L_2 \text{ and } x \notin L_1 \cap L_2 \}$

* $Prefix(L) = \{ prefix(x) \mid x \in L \}$

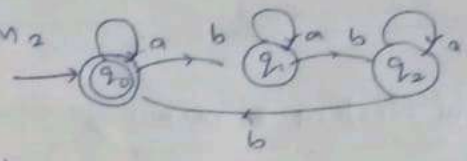
L_1 = { Strings with a's divisible by 2 }

L_2 = { " " b's " " 3 }

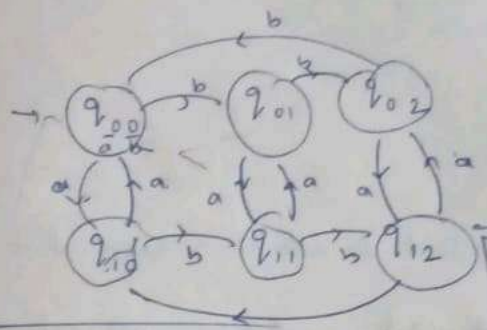
M_1



M_2



$L_1 \cup L_2$

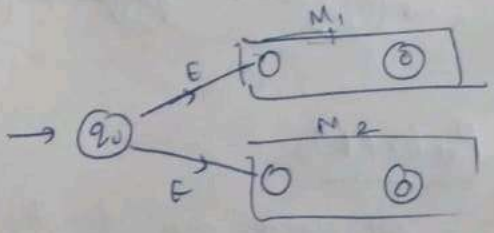


DFA & DFA.

	final states
$L_1 \cup L_2$	$q_{00}, q_{01}, q_{02}, q_{10}$
$L_1 \cap L_2$	q_{00}
$L_1 - L_2$	q_{01}, q_{02}
$L_2 - L_1$	q_{10}
$L_1 \Delta L_2$	q_{01}, q_{02}, q_{10}

* This is construction of DFA from DFA

NFA & NFA

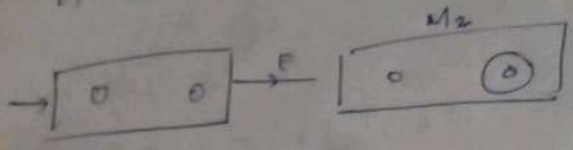


Epsilon transition with initial states

→ still finite

→ so it is regular.

$L_1 \cdot L_2$



intersection & minus ?

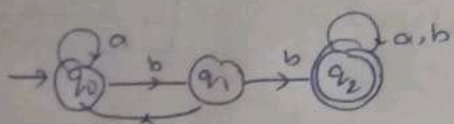
complement :-

L^c — ?

L^c : { string that do not have consecutive b's }

L : { strings that contains consecutive b's }

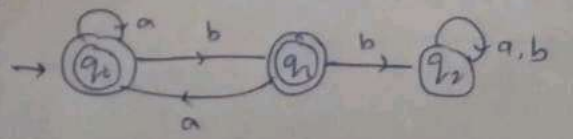
DFA



→ complement.

DFA:

L^c convert final as non-final, non-final as final



Prefix (L) :-

All states are final states except dead states

Reverse (L) :-

Initial → final final → Initial * transition direction changes (reversed)

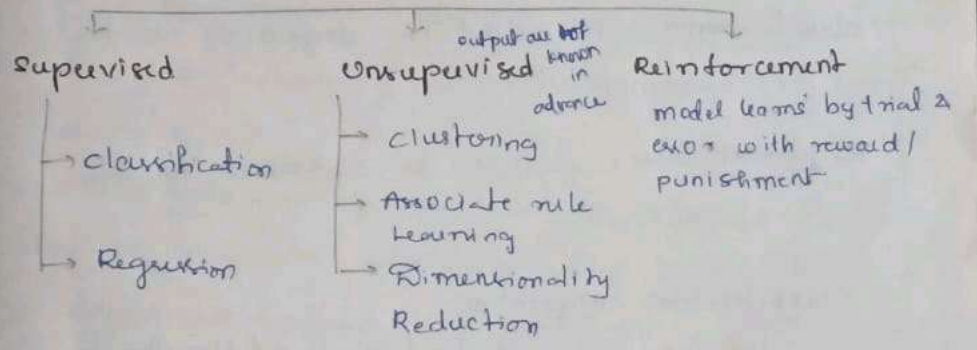
* remove the dead state

Machine learning

The process of making the machines to learn about patterns is called Machine learning
(It helps to automate decision-making from experience (data)
3-types of ML Algorithms.

1. Supervised learning trains model based labeled data
2. Unsupervised learning: trains model based on unlabeled data
3. Reinforcement learning. It has Agent (like Robo) & Environment (maze)

Types * rather than explicitly being programmed



I. Supervised learning:

Labels: Spam & not Spam.

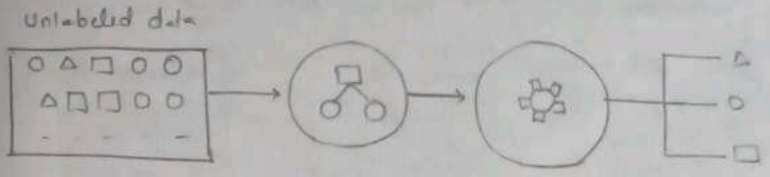
It takes labels as Input and trains model then predicts output.

For each Input corresponding output will be available Machinepredicted output is compared to actual output

→ This can be implemented through classification & Regression

II. Unsupervised learning:

→ Trains a model based on unlabelled data
→ It donot have any predefined output
→ It groups/ forms clusters of similar data
(Groups unlabeled data based on similarities)



→ Clustering: Grouping unlabelled data based on similarities
Ex: hierarchical clustering, k-mean clustering, Density-based clustering.

→ Associate rule Learning: rule based ML technique which is used to discover relationship blw large datasets
Ex: Appropriate Alogorithm, FP-growth Alg.

→ Dimensionality Reduction: It simplifies complex data so it can be easier to analyze & visualize data
Ex: PCA (Principle component analysis)
LDA (Linear Discriminant Analysis)

ML learning application:

III. Reinforcement learning

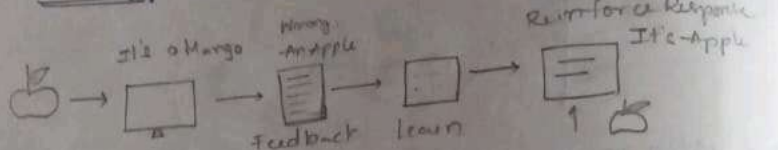
It consists of Agent & Environment.

→ Agent is decision maker (makes predictions)

→ Environment will give rewards or penalty.

→ process of training Agent to make decisions/ learn by interacting with Environment

→ Strategy: Maximize rewards to achieve goals



→ Here Agent will learn using trial & error method where Environment will respond through reward & penalty

State: current status of an Agent

In Robot in maze (Robot: Agent; Maze: Env)

Action: decision taken by Agent.

Reward:

If decision is correct, the Agent gets new state

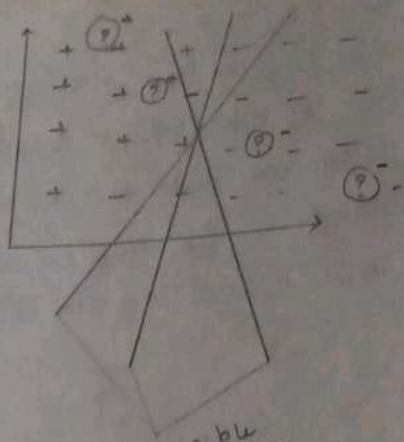
This continues till Agent reaches goal

Semi-Supervised Learning: ex: Audio recognizing

Few amt of labelled & more amt of unlabelled data.

Self Supervised learning: Model creates the labels by its own

Ex: Prediction of missing words in a sentence



From hypothesis space, the model can choose anyone accurate hypothesis from possible hypothesis

* Inductive bias:

It's a set of assumptions that learning Algorithms uses to make predictions on unseen or new data.

2 types

1) Language / Restriction bias (rules on hypothesis)
2) Preference bias (Prefer only simple models)

Language Bias:

In this, the model is only allowed to look at certain kind of hypothesis

Ex: only use straight lines, only use simple rules

Preference Bias:

Also called as "Search Bias". ^ Model It 'll consider all the hypothesis, but prefer some over other.

Ex: prefer simple models, smooth curves.

Evaluation:

Two types of evaluation metrics

① Classification metrics

② Regression metrics

confusion matrix: (modal performance will be evaluated by confusion matrix)

→ relationship between actual & predicted values

Actual \ Predicted	Predicted +ve	Predicted -ve
Actual +ve	TP	FN
Actual -ve	FP	TN

TP: correctly predicted the actual positives

TN: correctly predicted the actual negatives

FP: falsely " " actual ~~positives~~ negative

FN " " " actual ~~negatives~~ positive

3×3 confusion Matrix (for multi-classification)

Actual / ~~predicted~~	Predicted Sunny	Predicted cloudy	Predi. Rainy
sunny	70(TP)	5	2
cloudy	3	15(TP)	7
rainy	1	2	10(TP)

Sunny :-

TP = 70

FP = 4 (predicted as sunny) [3+1]

FN = 7 [5+2]

TN = 15+10+7+2 = 34

- Accuracy$_{sunny}$ = $\frac{TP+TN}{TP+TN+FP+FN}$ = $\frac{70+34}{70+4+7+34}$
- Precision$_{sunny}$ = $\frac{TP}{TP+FP}$ = $\frac{70}{70+4}$ = $\frac{70}{74}$
- Recall$_{sunny}$ = $\frac{TP}{TP+FN}$ = $\frac{70}{70+7}$ = $\frac{70}{77}$.

Rainy

TP = 10

FP = ~~5+2=7~~ 7+2 = 9

FN = 2+1 = 3

TN = 70+5+3+15 = 93

cloudy

TP = 15

FP = 5+2 = 7

FN = ~~3+7~~ = 3+7 = 10

TN = ~~3+7=10~~

70+2+1+10 = 83

Accuracy = $\frac{(TP_S + TP_R + TP_C) + (TN_S + TN_R + TN_C)}{(All\ TP) + (All\ TN) + (All\ FP) + (All\ FN)}$

* Macro-avg Precision = ~~Average~~ sum of precision / Total,

* Weighted-avg Precision = $\frac{\sum_{i=1}^{n} (\text{Precision} * \text{support})}{\text{overall Total}}$

$$\frac{17 + 6.63 + 73.15}{115} = \frac{96.78}{115} = 0.841$$

We can use individual metrix instead of classification metrix

cross-validation: one of

It is used to measure how the model performance of model on unseen data.

They are divided into splits.

1. K-Fold c.v
2. Hold-out c.v.
3. LOO cv (Leave-one out cross validation)
4. Stratified k-Fold c.v

1.) K-Fold c.v :-

It splits data into k equal parts

(k-1) parts are used for training

1 part is used for testing } for each fold,

Ex: k=5 (5 folds)

80% training	20% testing

(5-25) training. (1-5) testing

If modal is very simple & learns only few patterns are learnt.

* Bias: It is the error that happens when the ML modal is too simple & doesn't learn enough details (patterns) from the data.

→ High bias leads to underfitting.
→ It performs poor on both training & testing data.

* Variance: ML Modal It'll learn too much details including random noise.

→ High variance leads to overfitting.
→ It performs good at training data but poor at testing data.

* Overfitting:-

when the modal learns too much details including random noise.

Ex:

reasons: If modal gives high variance & low bias.
- If the modal is too complex.
- If the size of training data is too large.

* Underfitting:-

It happens when the modal is too simple to capture the underlying patterns from the data.

→ performs poor at both testing & training data.

reasons:- • If the modal is too simple
• High bias & low variance

* Linear Regression:- supervised algorithm | statistical technique

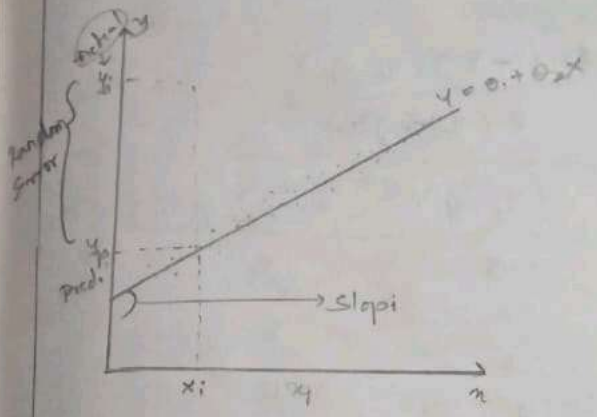
→ It is mainly used to model the relationship blw one or more independent variable (i/p) & dependent variable (o/p) by fitting the best possible st. line to data.

$Y = MX + C$

$Y = \theta_0 + \theta_1 x$

→ The main aim is to minimize the error by fitting the st. line. (diff. blw actual & predicted)

Assume that this is the best possible line



* To minimize the error, To find the best fit line, we use a method called least squares, this is used to minimize the sum of squared difference blw actual & predicted. This difference is known as residual.

$\text{Residual} = y_i - \hat{y}_i$

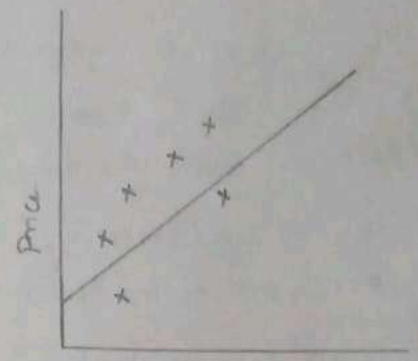
actual predicted

$SSE (\text{sum of squared error}) = \sum (y_i - \hat{y}_i)$

• Simple Linear Regression:-

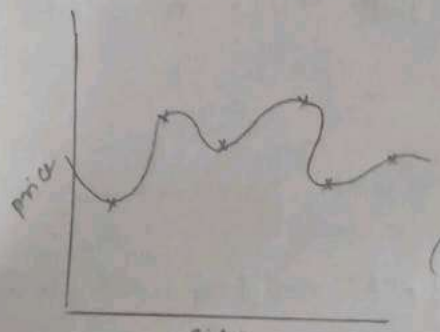
→ When we want to predict a target / depend. / o/p variable using only one feature means independent variables

$y = \theta_0 + \theta_1 x$



(underfitting)
high bias, low variance

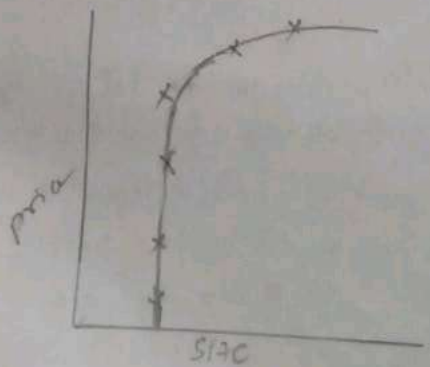
ex: Actual: 50 lakhs
predicted: 20 lakhs



(overfitting)

• touches all point including noise

(high variance, low bias)



(Appropriate / Good fit)

The curves that follow the data trend without over complicating to capture the patterns in the data

(low bias & low variance)

Performance on training data → very good
" " testing " → poor

Ex:

Fold	Training data	Test data
10	[20, 30, 40]	[10]
20	[10, 30, 40]	[20]
30	[10, 20, 40]	[30]
40	[10, 20, 30]	[40]

* Stratified K-Fold cross-validation:-

Each fold has same percent of each class as an original dataset. It is mainly used when classes are imbalanced.

Sample	class	class Distribution
A	0	class -0 → 2
B	0	class -1 → 3
C	1	
D	1	
E	1	

K=2

Fold	Class 0	Class -1
1	A	C, D
2	B	E

x_i	y_i	$\hat{y}$	residual
1	30	28	2
2	35	35.5	-0.5
3	40	43	-2.
4	50	50.5	-0.5
5	60	58	2.

Multilinear Regression:

It is procss of predicting dependent /out put variable using more than one independent /input variable

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n + \varepsilon$$

β_0 ↓ intercept.

$\beta_1, \beta_2, \beta_3 \ldots \beta_n \rightarrow$ slope / coefficients

$x_1, x_2, x_3 \ldots x_n \rightarrow$ input / independent variable

$y \rightarrow$ dependent variable.

Polynomial Regression:-

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_n x^n + \varepsilon$$

(Graph will be curve)

$n \rightarrow$ degree of polynomial.

$\beta_1, \beta_2, \beta_3 \ldots \beta_n \rightarrow$ Coefficient

$x \rightarrow$ input

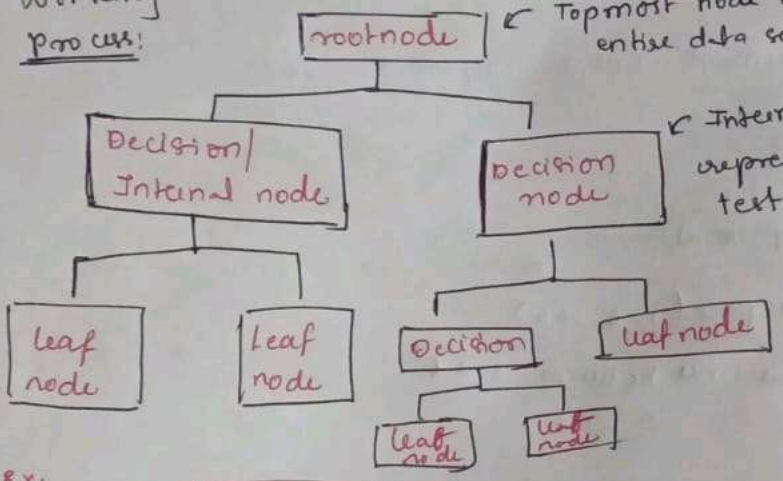
$y \rightarrow$ output

$\varepsilon \rightarrow$ Error

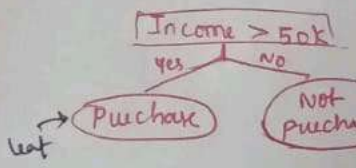
UNIT-II Decision-Tree

Splitting is based on feature selection

Working process:



Ex:



Branches: It represents attribute values. It'll connect the nodes.

Leaf node: It is a final node that do not split further that represents the final decision/Prediction.

Ex:



tree-1

Attribution selection measures:-

① Information Gain

② Gini Index.

Info^n Gain: How much the uncertainity decreases after the split.

Formula:

$$Gain(S,A) = Entropy(S) - \sum_{v}^{a} \frac{|S_v|}{|S|} \cdot Entropy(S_v)$$

← probability of s

S: set of instances

a: An attribute.

S_v: subset of s

(v represents an individual value that the attribute 'A' can take)

Entropy: It is the measure of uncertainity of a random variable

Formula:

$$Entropy(S) = -\sum_{i=1}^{n} P_i \log_2(P_i)$$

Ex:

class	count
yes	3
No	3

$P(yes) = \frac{3}{6} = 0.5$

$P(NO) = 0.5$

Entropy(S)

$$= -[0.5 \log_2 0.5 + 0.5 \log_2 0.5]$$

$$= 1$$

* Entropy is zero if data is of same class, but 1 if it is mixed

(child-1)

when $x = 1$

class I = 2

class II = 1

$\frac{2}{3}$ & $\frac{1}{3}$

$$\text{Entropy}_{\text{child}_1} = -\left[\frac{2}{3}\log\frac{2}{3} + \frac{1}{3}\log\frac{1}{3}\right]$$

$$= +[0.918]$$

(child-2)

when $x = 0$

class II = 1

$$\text{Entropy}_{\text{child}_2} = -[1\log 1 + 1\log 1] = 0$$

$$= 0$$

Parent entropy
↓
splitting
↓
Individual entropy
↓
weighted entropy

child-1 child-2

$$\text{Weighted Entropy} = \frac{3}{4} \times 0.918 + \frac{1}{4} \times 0$$

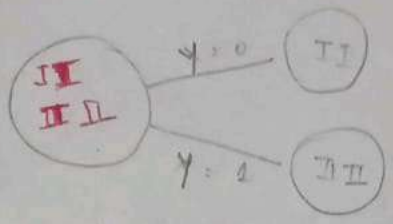
$$= 0.6885$$

weighted entropy: $\sum \frac{\text{Individual of child}}{\text{Total instances}} \times$ child entropy

Information Gain (x): 1 − weighted entropy

$\text{Gain}(x) = 1 - 0.688 = 0.311$

For y →



class I = 2 $\frac{2}{4} = 0.5$

class II = 2 $\frac{2}{4} = 0.5$

$Entropy_{parent} = 1$

when y = 1

class I = 2

class II = 0

$Entropy_{child_1} = 0$

when y = 0

class I = 0

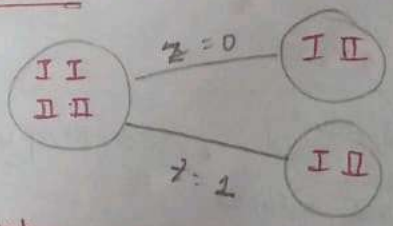
class II = 2

$Entropy_{child_2} = 0$

Weighted entropy = 0

Gain (y) = 1 − 0 = 1

For z



$Entropy_{parent} = 1$

when z = 1

class I = 1 = 1/2 = 0.5

class II = 1 = 1/2 = 0.5

$Entropy_{child_1} = 1$

when z = 0

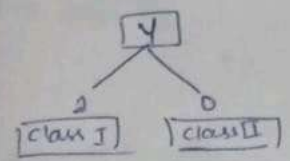
class I = 1 ⇒ 1/2

class II = 1 ⇒ 1/2

$Entropy_{child_2} = 1$

Weighted entropy = ½ × 1 + ½ × 1 = 1

Attribute with lowest weighted Gini is considered
Since 'Y' has least weighted Gini Index



Appropriate Problems for Decision-Tree Learning

It should consist below characteristics

1) Attribute value pair
2) Discrete or continuous output variable
3) Disjunctive (OR) descriptions.
4) Presence of errors & missing values.
5) Interpretable models

Attribute Value Pair:-

If each attribute is associated with value. then that problem is best suited for decision-tree learning.

Ex:

outlook	Humidity	play
sunny	high	yes
rainy	Normal	No.

Discrete or continuous output variable:-

If the output variable is discrete like Yes/No (True/False) (Pass/Fail) comes under classificatio
If continuous then regression (ex: house price predictio

3.) Disjunctive Description.

If the target variable is expressed based on multiple 'OR' conditions then best suited for Decision tree

Ex:

if (Income > 50K) or (discount available) or (previous purchase)

4) Presence of Errors & Missing values:-

Even though data contains Errors & missing values, It will handle them & do appropriate & accurate predictions.

5) Interpretable Models:-

In this prestructure is easy to visualize & interpret which is a major advantage in the application where understanding the reasoning behind a decision is crucial.

Ex: Medical diagonosis, loan Approval.

Here Decision Making tree should be transparent & make appropriate prediction.

Basic Decision Tree Algorithm :- ID3

ITERATIVE DICHOTOMISER (ID3)

All the training instances should be kept in root node and splitting is done

Tree pruning:- To avoid over-fitting we use Tree pruning

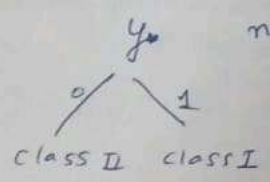
In ID3, Generally it doesn't need as its work is to remove unnecessary branches.

Ex:

1.) Start (4)

2.) Select Best Attribute ("y.") (Gain(x) = 0.311
Gain(y) = 1 → select
Gain(z) = 0)

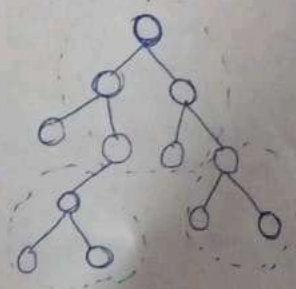
3.) Creating Tree node



4.) Stopped.

Hypothesis space search in Decision Tree learning

→ Set of all possible DT's that can be formed using attributes (Given Dataset)



→ ID3 searches this space to find a tree that correctly classifies the training data. & Attribute selected.

→ It uses Hill-climbing Approach

→ It starts with an empty Tree

Limitations

1) No Back-Tracking is available. once ID3 selects an attribute at a node. It never reconsiders it.

2) It keeps only one current tree during learning or Training (Instead of all subtrees)

Advantages / capabilities:

1) It use all training data at every state. ID3

2) At uses all training samples to select the best split.

Inductive Bias:-

It determines which Decision tree ID3 selects.

- set of assumptions of learning diagram to generalize beyond the training data.

Two types:

① Approximate Inductive bias of ID3:- It selects smaller trees compared to larger trees

② A closure Approximation to the Inductive bias of ID3:-

① Trees with smaller structure compared to large tree

② In that it'll calculate Info^n Gai

2nd step: Trees that place high Info^n gain attribute close to the root are preferred.

Issues in the Decision Tree:-

① Overfitting:-

To overcome the overfitting we use tree-pruning

Soln:

→ Pruning: cut-off the unnecessary branches
- limit the depth.
- min samples decided

* Pre-Pruning:
- Lower info-gain features are ignored

*. Post-Pruning: After the completion of Decision-tree, feature with least Accuracy will be cut-off.

→ Limit the depth of tree

→ Assemble Methods:- combining multiple trees & random forest to overcome the prob. of overfit

② High Variance:- If any small modif^n done in training data, it will lead to significant change in predictor & structure.

Soln:-

- Random forest & Gradient Boosting:- To aggregate the outputs of multiple tree to overcome High variance

③ Bias:- The Attribute selection measures (Info gain/Gini Index) biased towards feature with large no. of unique values that dominates the actual data set

soln:-
- Balanced data set
- feature with more unique value should be removed.

(4) Greedy Algorithm:-
It uses a greedy Algorithm to make decisions at each step but it may not lead to the best overall tree structure.

soln:
- Hyperparameter tuning / Pruning.

(5) Face Difficulty with Linear relationship:-

soln:
- SVM or Linear Regression can be used.

Instance based learning:-
① It memorizes data
② new data comes
③ compares with old data for prediction

Lazy learner Algorithm.

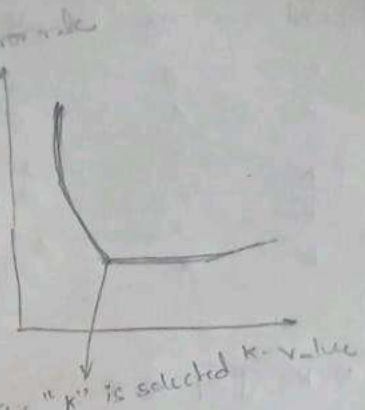
It works only whenever predictions are needed based on Similarity.

If new data point comes for prediction, It will compare the data to the stored data to make prediction based on similarity

- KNN & Case Based Reasoning.
- k-Nearest Neighbours:-
  Imp. Used for classification & Regression
  - The new data point is classified based on the majority class of its nearest neighbours.

③ Elbow Method:-

Error rate gradually decreases as k-value increases at a certain point error rate starts increasing → That point is taken as k-value.



② Better to choose odd 'k' values → why?

④ $k \approx \sqrt{n}$ → selected

KNN Algorithm :-

- step-①: selecting optimal k-value
- step-②: calculate the euclidean distance

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- step-③: Find k-nearest neighbours → sorting
- step-④: Make prediction

classification:- majority of a particular class is selected

Regression:- It take the avg of k-neighbours

→ Equal distances leads to difficulty in prediction of class which is a disadvantage.

Curse of Dimensionality:-

More Dimen ↑ → Data sparse → nearest neighbour find (difficult) → prediction difficult

It refers to the problems that arise when we're working with the data that has large number of features

As Dimension increase, Data becomes sparse, it is difficult for ML Algorithm to find out the pattern.

* Feature extraction (PCA)
* Data Preprocessing.

Effects:

- overfitting — sometime Equal distances when data becomes sparse
- Increased computational compuxity
- Longer training time.

Solutions:-

1) **Dimensionality Reduction:-**
   - i) Feature selection → [Relevant features will be selected, irrelevant " " ignored] whichever giving best performance will be selected.
   - ii) Feature extraction → converts High Dimen. Features to low " " Done by PCA (Principal component Analysis)
2) **Pre-Processing**

   There are
   1. Data normalization
   2. Fill missing values should be handled by removing or filling by mean, median, mode.

steps in F.S :-

step-1: start with empty Feature set

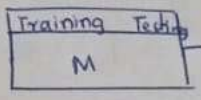
step-2: evaluate all features one at a time by training a model with one feature

step-3: select the feature that gives the best performance then add that feature to Feature set.

step-4: repeat until stopping criteria is met

Ex: d → 100 features

$f_1 \dots f_{100}$. empty $\{f_{10}\}$

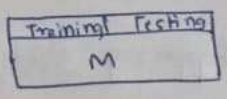
$f_1 \longrightarrow$ 

→ performance a_2

$f_2 \longrightarrow$ " → " a_2

$f_{10} \longrightarrow$ " → " → a_{10}

$f_{100} \longrightarrow$ " " a_{100}

$\{f_{10}, f_1\} \longrightarrow$ 

→ accuracy → b_1

$\{f_{10}, f_2\} \longrightarrow$ " → " b_2

$\{f_{10}, f_{10}\} \longrightarrow$ " → " b_{10} → best

$\{f_{10}, f_{100}\} \longrightarrow$ " → " b_{100}

$\{f_{10}, f_{50}, \dots\}$

Ex: House price prediction

x_1 = Area , x_2 = no of bedrooms , x_3 = Distance from city

x_4 = Age of the house

	RMSE
x_1	90
x_2	98
x_3	50
x_4	99

select

$\{x_2, x_3\} \rightarrow 90$

$\{x_1, x_3\} \rightarrow 98$ — select

$\{x_1, x_4\} \rightarrow 51$

$\{x_1, x_3, x_2\} \rightarrow$

$\{x_1, x_3, x_4\} \rightarrow$

If same RMSE occurs then stop the process & take only $\{x_1, x_3\}$

Steps in BST

Step-1: Starts with all features

Step-2: Remove one feature at a time. Train the model on remaining features & calculate the performances

Step-3: Repeat until stopping criteria occurs

Ex: $\{x_1, x_2, x_3, x_4\}$

Removed	Remaining	
x_1	$\{x_2, x_3, x_4\}$	calculate performance →
select x_2	$\{x_1, x_3, x_4\}$	" → best
x_3	$\{x_1, x_2, x_4\}$	" →
x_4	$\{x_1, x_2, x_3\}$	" →

features	χ^2 score
② Coaching class	14.06
① Attendance > 75%	19.20
③ Sports participation	1.50
④ Hostel Resident	0.50

Top-k if $k = 2$ we take two features coaching & Attendance.

* Multivariate feature selection:-

It evaluates multiple features together, not independently.

① Frame the model
② measuring overall performance
③ selection or removal features based on combined effect.

Advantage:-

- It removes redundant or highly correlated features
- It captures other feature Interactions.

Disadvantage:-

- computationally more expensive.

Techniques

1) Filter method.
   - correlation matrix

2.) Wrapper Method
   - recursive feature Elimination.
   - Forward f. Seln ∘ Back. F. Seln.
   → 3 techniques

3) Embedded Method
   ∘ Lasso Regression.

4) Dimensionality Reduction
   - PCA.

1) Filter Methods

We need to find out correlation among multiple features.

- correlation Matrix

house	Area (x_1)	Rooms (x_2)	Area-m^2 (x_3)
1	1000	3	93
2	1200	4	112
3	1400	5	130
4	1600	6	149

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

$W = \begin{bmatrix} 0.86 \\ 0.64 \\ 0.04 \end{bmatrix}$ → study hour, → Attendance, → social media.

Highest will be considered. lowest will be removed

To calculate intercept

$b_0 = \bar{y} - (w_1\bar{x}_1 + w_2\bar{x}_2 + w_3\bar{x}_3)$

2.) Losso Regression

It is also known as L_1-Regularization.

L_2 Regression
Ridge

Here we use penalty term "lambda" (λ)

Penalty = it acts cost for non-zero coefficients

$$\text{Loss} = \text{MSE} + \lambda \sum_{j=1}^{n} |w_j|$$

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y})^2$$

$\hat{y} = b_0 + w_1 x_1 + w_2 x_2$

x_1 (study hours)	x_2 (social media)	y (score)
2	4	2
2	5	2
3	5	4
3	5	6

$$MSE = \frac{1}{3}\left[(2 - w_1(1) - w_2(5))^2 + (4 - w_1(2) - w_2(5))^2 + (6 - w_1(3) - w_2(5))^2 \right]$$

here $\lambda = 1$

$w_1 = 1$, $w_2 = 0$

$MSE_2 = 0$

loss = MSE + $w_1 + w_2$

loss => $0 + [w_1 + w_2]$

loss => $0 + 0 + 2$

loss => 2

- highest values needs to be considered.

  ↓

  There will be a threshold value, above which values are highest and can be considered. (Those features can be considered)

- If lowest values occurs, we need to change the lamda (i.e, coefficient value (λ) needs to be changed)

$$r = \frac{n(\Sigma xy) - (\Sigma x)(\Sigma y)}{\sqrt{[n\Sigma x^2 - (\Sigma x)^2][n\Sigma y^2 - (\Sigma y)^2]}}$$

r(Area, Area m²) = 0.999

↓ Highest correlated features, so we're removing either one of it which is Area-m² (based on all combinations)

2) Wrapper Method :-

- Recursive feature Elimination

It is based on metric

Ex:

student	study hours (x_1)	Attendance (x_2)	Social Media (x_3)	Exam Score (y)
1	2	60	5	50
2	4	70	4	65
3	6	80	3	80
4	8	90	2	90

$$y = b_0 + w_1x_1 + w_2x_2 + w_3x_3$$

$$X = wx$$

$$w = (X^T \cdot X)^{-1} X^T \cdot Y$$

$$X = \begin{bmatrix} 2 & 60 & 5 \\ 4 & 70 & 4 \\ 6 & 80 & 3 \\ 8 & 90 & 2 \end{bmatrix}_{4\times3} \qquad Y = \begin{bmatrix} 50 \\ 65 \\ 80 \\ 90 \end{bmatrix}_{4\times1}$$

Ex:

coaching-class	Pass	Fail.	Row Total
Yes	45	15	60
No	15	25	40.
	60	40	100

$R_1C_1 = \frac{60 \times 60}{100} = 36$

$R_1C_2 \quad \frac{60 \times 40}{100} = 24$

$R_2C_1 = \frac{40 \times 60}{100} = 24$

$R_2C_2 = 4 \times 4 = 16$

O_i	E_i	$O_i - E_i$
45	36	9
15	24	-
15	24	
25	16	

$\{x_1, x_3, x_4\}$

$x_1 \longrightarrow \{x_3, x_4\}$

$x_3 \longrightarrow \{x_1, x_4\}$

$x_4 \longrightarrow \{x_1, x_3\}$ ~~40~~ → best

$\{x_1, x_3\} \rightarrow 41$ (same / no much improvement)

stop

* Univariate Feature selection:—

It evaluates each feature independently against the target variable and it selects the features based on individual test.

① calculate the score
② rank the feature
③ select top 'k' features

Advantage.

① computationally fast
② easy to implement
③ Good for Dimensionality reduction.

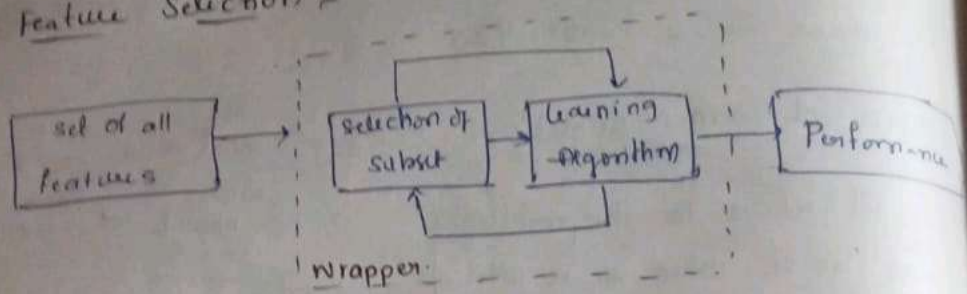
Disadvantage:

① It won't consider other feature interaction.

To perform this, we use chi-square Test (categorical variable)

& Anova Test.

Feature Selection :-



Stopping criteria

① Performance decrease / improvement

② No big change in performance by adding more features

③ If all features are finished/reached.

1) Forward Search (Forward Feature Selection) :-

- Starts with empty set. It adds 1 feature at a time.
- Greedy wrapper based approach.

2.) Backward Search (Backward feature Elimination) :-

- Starts with All feature set.

* It takes more time to search compared to forward search

point	x	y	class
A	2	3	class 1
B	4	5	class 1
C	5	2	class 1
D	6	5	class 2
E	7	6	class 2
F	8	5	class 2

$n = 6 \times 4 = 24$
↓
no. of rows & col

1) $K = \sqrt{24} \approx 5$

Target point (new data point) = (5, 4)

2) A(2, 3) T(5, 4) $d_A = \sqrt{(5-2)^2 + (4-3)^2} = \sqrt{10} \Rightarrow 3.162$

B(4, 5) $d_B = \sqrt{(5-4)^2 + (4-5)^2} = \sqrt{2} = 1.414$

C(5, 2) $dC = \sqrt{(5-5)^2 + (4-2)^2} = \sqrt{2^2} = 2$

D(6, 5) $d_D = \sqrt{(5-6)^2 + (4-5)^2} = \sqrt{2} = 1.414$ 1.5

E(7, 6) $d_E = \sqrt{2^2 + 2^2} = \sqrt{8} = 2\sqrt{2} = 2.828$

F(8, 5) $d_F = \sqrt{3^2 + 1^2} = \sqrt{10} = 3.162$ 3.10

Sorting

B → 1
D → 2
C → 1
E → 2
F → 2
A → 1

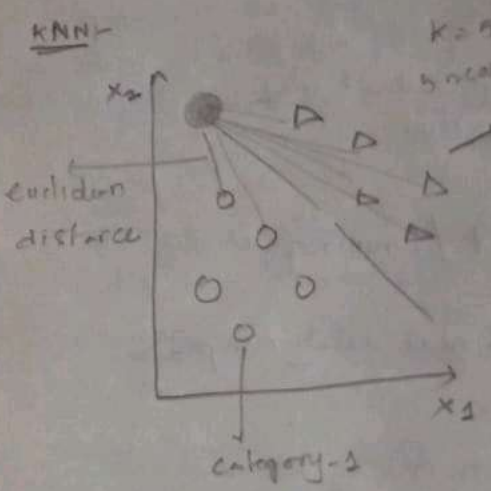
$k = 5$

class-2 → D, E, F, →
class-1 → B, C

∴ T(5,4) belongs to class-2

It won't learn from training data immediately so it is also called as Lazy learner Algorithm.
(only when prediction is needed)

KNN:



1 → 3 (nearest neighbours from class 1)
2 → 2 (" " " " 2)

→ majority of neighbours are more in category-1.
so new datapoint is predicted or sent to category-1

* Selection of k-value is important.
(k → no. of nearest neighbours)

Small k-value ⟶ overfitting
Larger k-value ⟶ Underfitting

* Selection Methods of k-Value:-

① Cross validation:- Best k-value is selected
→ ① Divide into k-splits
② k-1 → Training 1 → testing
③ Best k value is selected

① Start
② Selection of Attribute
③ creating a DT
④ Stop (criteria: → If same class
→ No splitting forward further)

STEP-①
Start:- Input the training dataset containing set of Attributes & labels. Place all these at root node.

STEP-②
Select the Best Attribute:-
→ we should calculate entropy for each attribute
→ Info^n Gain or Ginni Index is calculated.
→ Max Gain or min GI is selected

STEP-③
Creating Tree node:-
create a tree node in Decision Tree for the selected attribute. Split the dataset into subsets. For each subset ID3 selects the next best attribute using recursive process.

STEP-④
Stopping Criteria:- the recursion continuous until one of stopping criteria is met.
① If no ~~further~~ Attribute remains for further splitting.
② When all the instance are belong to same class

STEP-⑤
Handling Missing values:-

STEP-⑥ Post Pruning

Weighted Ginni$_{(x)}$ = $\left(\frac{3}{4}\right) \times 0.44 + \left(\frac{1}{4}\right) \times 0$

$= 0.33 + 0$

Weighted Ginni$_{(x)} = 0.33$

Split on y:

$y = 1$ class I = 2, class II = 0. Ginni$(y=1) = 1 - \left(\left(\frac{2}{2}\right)^2 + \left(\frac{0}{2}\right)^2\right)$

$= 1 - 1 = 0.$

$y = 0$ class I = 0, class II = 2 Ginni$(y=0) = 1 - \left(\left(\frac{0}{2}\right)^2 + \left(\frac{2}{2}\right)^2\right)$

$= 1 - 1 = 0.$

Weighted Ginni$_{(y)} = \left(\frac{2}{4}\right) \times 0 + \left(\frac{2}{4}\right) \times 0 = 0.$

Weighted Ginni$_{(y)} = 0$

Split on z:

$z = 1$ class I = 1, class II = 1 Gini$(z=1) = 1 - \left(\left(\frac{1}{2}\right)^2 + \left(\frac{1}{2}\right)^2\right)$

$= 0.5$

$z = 0$ class I = 1, class II = 1 Ginni$(z=0) = 1 - \left(\left(\frac{1}{2}\right)^2 + \left(\frac{1}{2}\right)^2\right)$

$= 0.5$

Weighted Ginni$_{(z)} = \frac{2}{4} \times \frac{1}{2} + \frac{2}{4} \times \frac{1}{2}$

$= \frac{1}{4} + \frac{1}{4} = 0.5$

Weighted Ginni$_{(z)} = 0.5$

Gain (z) = 1 - 1 = 0

Hypothesis Space

Gain (x) = 0.311

Gain (y) = 1 ⟶ Best one

Gain (+) = 0

Based on y ⟶ It'll do the process

Gini Index: It's a metric to measure how often a randomly choosen element would be incorrectly identified.

*Attribute with lowest Gini Index should be preferred

Formula = $1 - \sum_{i=1}^{n} P_i^2$ $P_i \rightarrow$ Probability.

Either we Information Gain or Gini Index to select the attributes

Total Instances = 4
class I = 2
class II = 2

$Ginni_{Parent} = 1 - \left(\left(\frac{2}{4}\right)^2 + \left(\frac{2}{4}\right)^2 \right)$

$= 1 - 0.5$

$= 0.5$

Split on x

x = 1, class I = 2
class II = 1

$Ginni(x=1) = 1 - \left(\left(\frac{2}{3}\right)^2 + \left(\frac{1}{3}\right)^2 \right)$
$= 0.44$

x = 0, class I = 0
Class II = 1

$Ginni(x=0) = 1 - \left(\left(\frac{1}{1}\right)^2 + \left(\frac{0}{1}\right)^2 \right) = 1 - 1 = 0$

* If high entropy when data is mixed & Low when data is pure (no confusion, clear)

* ~~En~~ tropy is zero if all samples belongs to Same data

* The Attribute which is having the highest Info gain will be given

X	Y	Z	C
1	1	1	I
1	1	0	I
0	0	1	II
1	0	0	II

~~total~~ instances = 4

class I = 2

class II = 2

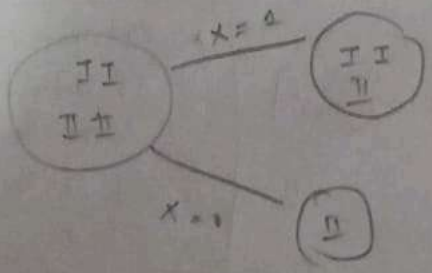
Entropy = 1

$P(I) = \frac{2}{4} = 0.5$

$P(II) = \frac{2}{4} = 0.5$

~~Entropy = - fo s log2 o s o r t~~

$Entropy_{parent} = -[0.5 \log_2 0.5 + 0.5 \log_2 0.5] = 1$



Regression Metrics:

1.) MAE = $\frac{1}{n}\sum_{i=1}^{n}|x_i - y_i|$

2) MSE = $\frac{1}{n}\sum_{i=1}^{n}|(x_i - y_i)^2|$

3) RMSE → $\sqrt{MSE}$

4) R-squared (R^2) → $1 - \frac{SSR}{SST}$ (Residual Error / SSE)

① MAE.

Actual	Predicted	Error	Abs Error	Squared Residual
10	12	-2	2	4
20	18	-2	2	4
30	33	-3	3	9
60	63		7	17
3	3		3	3
20	22		2.3	5.66

Σ SSR

MAE = 2.33 MSE = 5.66

RMSE = $\sqrt{MSE} = \sqrt{5.66}$

R^2 - squared ⇒ SST = SSE + SSR

SST = (actual $y_i - \bar{y}$) + (predicted $\hat{y} - \bar{y}$)

SST = $\Sigma(y - \bar{y})^2$

= ~~$\Sigma(12-20)^2 + (-$~~

= $(10-20)^2 + (36-20)^2 + (30-20)^2$ ✓

= $10^2 + 10^2 = 10^2 + 10^2 = 100 + 100 = 200$

R^2 ⇒ $1 - \frac{17}{200}$

= $1 - 0.085 = 0.915$

X (Exp)	Y (Salary) in thousands
1	30
2	35
3	40
4	50
5	60
15/5 = 3	215/5 $\bar{y} = 43$

$\hat{y} = \theta_0 + \theta_1 x$

$\theta_1 = \frac{cov(x,y)}{var(x)} = \frac{\Sigma (x_i - \bar{x})(y_i - \bar{y})}{\Sigma (x_i - \bar{x})^2}$

$(x_i - \bar{x})^2$	$(x_i - \bar{x})$	$(y_i - \bar{y})$	$(x_i - \bar{x})(y_i - \bar{y})$
4	-2	-13	26
1	-1	-8	8
0	0	-3	0
1	1	7	7
4	2	17	34
10			Σ 75

$\beta = \frac{75}{10} = 7.5$

$\alpha = \bar{y} - \beta \bar{x} \Rightarrow \alpha = 43 - (7.5)(3)$

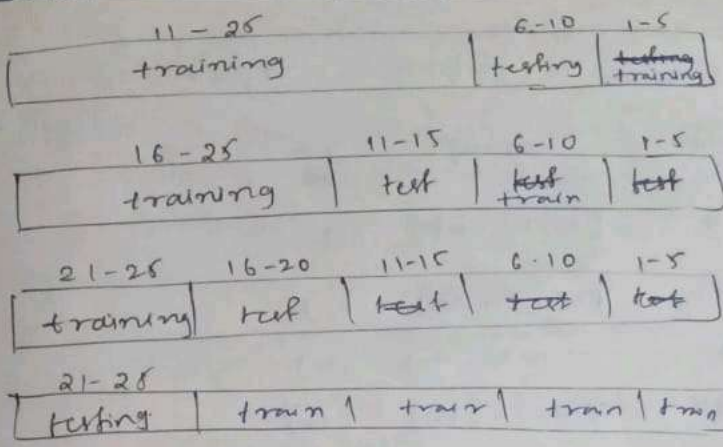
$= 43 - 22.5$

$= 20.5$ //

$\boxed{\hat{y} = 20.5 + 7.5x}$

residual : $\hat{y}_p = 20.5 + 7.5(1) = 20.5 + 7.5 = 28.0$

$\hat{y}_i - \hat{y}_p = 30 - 28 = 2.$



Iteration	training	testing.
1	[6-25]	[1-5]
2	[1-5, 11-25]	[6-10]
3	[1-5, 6-10, 16-25]	[11-15]
4	[1-5, 6-10, 11-15, 21-25]	[16-20]
5	[1-5, 6-10, 11-15, 16-20]	[21-25]

2.) Hold-out cross-validation:

50% → training · 50% → testing

→ model may miss patterns of other-half side which leads to high bias (Error)

→ If there is a high bias then model leads to underfit

3) LOOCV:

The model is trained on entire data except one data point.

→ It leads to overfitting → It'll take more time on

→ low bias & high variance large datasets.

Ex: covid test results (200 test results)

A/P	P+ve	P-ve
A +ve	80(TP)	20(FN)
A -ve	10(FP)	90(TN)

- False positive is also known as TYPE-I ERROR
- False Negative is also known as TYPE-II ERROR

-Accuracy: overall ~~Actual~~ correctness of overall model

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$$

Precision:- out of all positive's Quality of model's +ve outcomes

$$\text{Precision} = \frac{TP}{TP+FP}$$

Recall: out of all +ve cases, how many did the model correctly detected

$$\text{Recall} = \frac{TP}{TP+FN}$$

F2 Score: Harmonic mean of Precision+recall

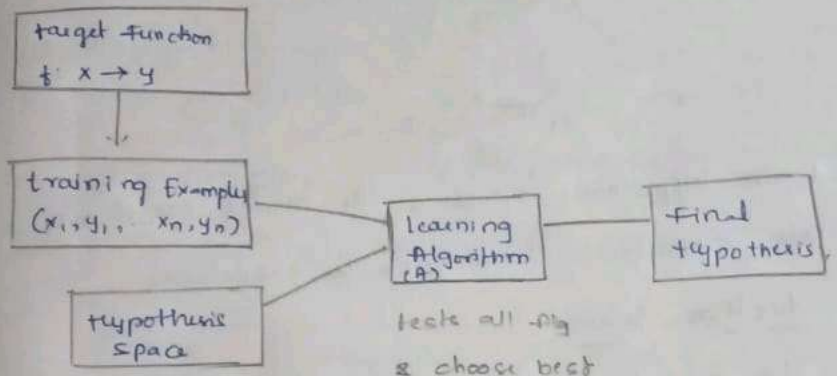
$$F_1 \text{ score} = 2 \times \left(\frac{\text{Precision} \times \text{recall}}{\text{Precision} + \text{recall}} \right)$$

* Hypothesis Space:

A hypothesis is a model's presumption regarding the connection blw the I/p features & O/p.

Ex: House price detection (based on I/p (house) features)

→ For each probability, we get many solutions but we need to choose accurate one (like size of house)



Hypothesis Space:

It's a collection of all possible hypotheses where learning Algorithm could choose

Ex:

If hypothesis is a straight line

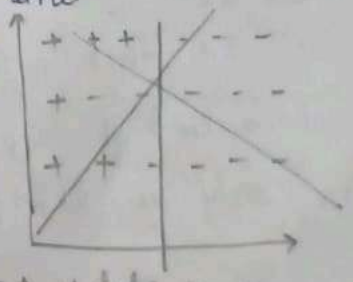
$Y = MX + L$

Y: predicted output

M: Slope

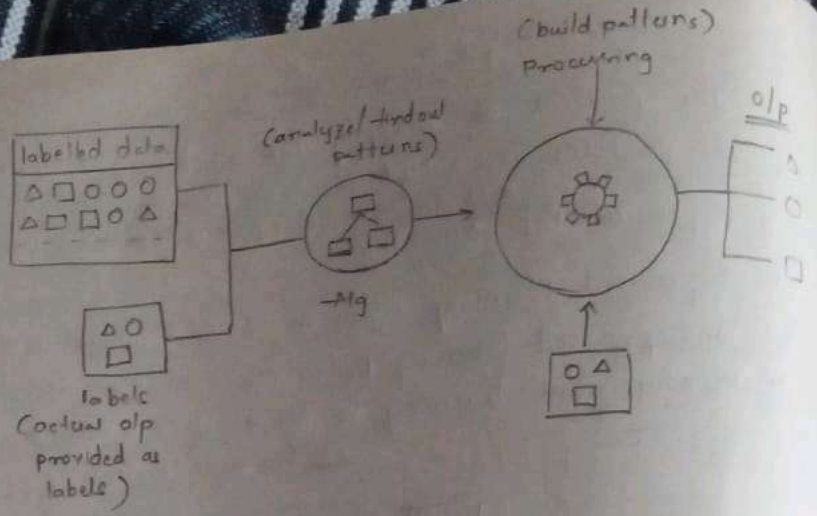
X: I/p

L: Y - Intercept



+ - + - : data Space

A straight line hypothesis can have many possible locations



Applying based on output

→ classification: when output is categorical variable
ex spam/not

→ Regression: when output is continuous variable
ex house cost

→ Classification Algorithm:
logistic Regression, Random forest Algorithm, support vector mach, naive bay, K-nearest neighbour, decision tree algorithm

→ Regression Algorithm: "learns the relationship between Input features & numerical target variable"

Ex: linear Regression, polynomial Regression, Random forest Regression, decision tree Regression

* Pumping lemma for R.L :- Language recurrent pattern

Let 'L' be a Regular Language then there exists an integer $n \geq 1$ depending only on 'L' such that every string 'x' in 'L' of length atleast n can be written as $x = uvw$ (partitioned 3 parts) satisfying

1. $|v| \geq 1$ (length of v → It shouldn't be empty)
2. $|uv| \leq n$
3. $\forall\ i \geq 0$ $uv^iw \in L$

This is ↓ Recurrent pattern.

Ex: a^{2n} here $v = a^2$

· complement:

$$L^c = \{ x \mid x \in \{ \Sigma^* - L \} \}$$

· reverse (L):

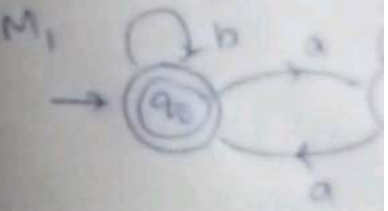
$$[L^R] = \{ x \mid x^R(x) \in L \}$$

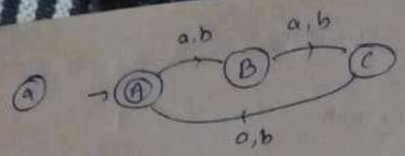
RL are closed under all three operator which means resultant is regular language

$L_3 = \{$ Strings

$L_2 = \{$

M_1





$A : \varepsilon + C(a+b)$ — (1)

$B : A(a+b)$ — (2)

$C : B(a+b)$ — (3)

Sub (2) in (3)

$C = A(a+b)(a+b)$ — (4)

Sub (4) in (1)

$A = \varepsilon + A[(a+b)(a+b)(a+b)]$

$A = \varepsilon[(a+b)(a+b)(a+b)]^*$

$A = [(a+b)(a+b)(a+b)]^*$

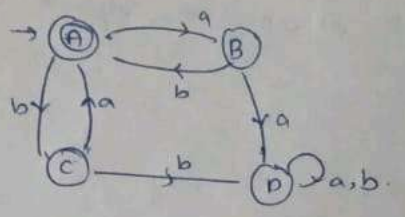
$[(a+b)^3]^*$

Conversion

* Regular Expression are equivalent expression for regular language

• If language is not regular than we cannot write regular expression

2 sol)



A : ε + Ca + Bb — (1)

B : Aa — (2)

C : Ab — (3)

Substitute (2) & (3) in (1)

A = Aba + Aab + ε

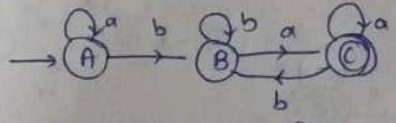
A = A(ba + ab) + ε [∵ Distr. property]

R = RP + Q
R = QP*

⇒ A = ε(ba + ab)*

A = (ba + ab)* (∵ r·ε = r) identity.

3 sol)



A : ε + Aa — (1)

B : Ab + Bb + Cb — (2)

C : Ba + Ca — (1)

Apply Lemma

A = εa* = a* (∵ (QP* & r·ε = r))

$$\frac{R = RP + Q}{A = Aba + Bb}$$

Aba + Aab

Bb·(ba)*

Aa(ba)*

A = Aa(ba)*

A = A(ba + ab) + R = R·P + Q

4 sol)

(14) Strings which do not consecutive b's.

$(ab+a)^* + b(ab+a)^*$

$(E+b)(ab+a)^*$

$(ba+a)^*(\epsilon+b)$

$b^*a^*b^-$

4 options

len-1: a, b, ε

len-2: a b, a a, b b, b a

(15) Multiple Reg Ex are possible

Algebraic Properties of RE:-

* Commutative Property:-

changing the order of operands doesn't change the results.

- regular expression satisfy C.P over union operation

$r_1 + r_2 = r_2 + r_1$

where r_1 r_2 are R.E's

- reg Ex do not satisfy C.P over concatenation operation

$r_1 r_2 \neq r_2 r_1$

* Associative Property:-

changing the order of operators or operations doesn't change the results

Regular Expressions — (Derivative)

For every regular language, we can write Regular expression

Operators used: Union (+) {Binary operators}
Concatenation (.) {Binary operators}
Kleene closure (*) {Unary}

R.E.S : 1) ϵ is a R.E
2) a is a R.E {a is I/p symbol}
3) a* is a R.E
4) a op b is a R.E

$a^* = \{\epsilon, a, aa, aaa, aaaa \ldots\}$

$a^* = a^0 + a^1 + a^2 + a^3 + a^4 + \ldots\ldots$

$a + b \Rightarrow \{a, b\}$

$a.b \Rightarrow \{ab\}$ (a followed by b) [pattern]

ab → 1 string of length 2

* → no. of repetitions

write the Regular Exp^n over alphabet {a,b} for the following

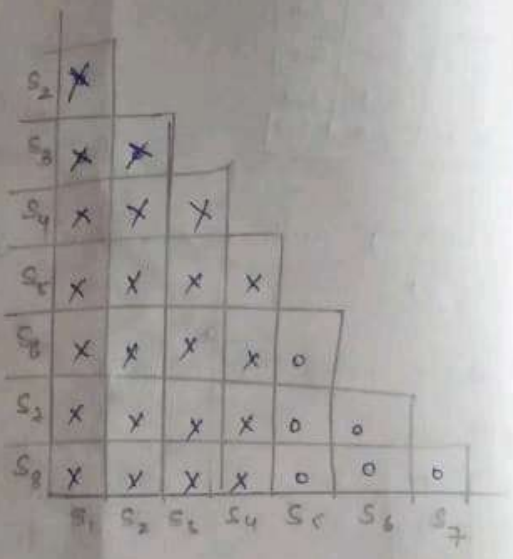
① All the strings

$(a+b)^*$

$L = \{\epsilon, a, b, ab, aa, ba, bb, \ldots, a \sim b \ldots\}$

② substring 1101

	0	1
S_1	S_1	S_2
S_2	S_1	S_3
S_3	S_4	S_3
S_4	S_1	S_5
*S_5	S_6	S_7
*S_6	S_6	S_5
*S_7	S_8	S_7
*S_8	S_6	S_5



final

$\{S_5, S_6, S_7, S_8\}$

$\{S_1, S_2, S_3, S_4\}$

○ $\{S_1, S_2\} \rightarrow$ on 1 $\{S_2, S_3\}$ $\{S_5, S_6\}$

$\{S_1, S_3\} \rightarrow$ on 0 $\{S_1, S_4\}$ $\{S_5, S_7\}$

① $\{S_1, S_4\} \rightarrow$ on 1 $\{S_2, S_5\}$ $\{S_5, S_8\}$

$\{S_2, S_3\} \rightarrow$ on 1 $\{S_1, S_4\}$ $\{S_6, S_7\}$

② $\{S_2, S_4\} \rightarrow$ on 1 $\{S_3, S_5\}$ $\{S_6, S_8\}$

③ $\{S_3, S_4\} \rightarrow$ on 1 $\{S_1, S_5\}$ $\{S_7, S_8\}$

④

~~$[S_1 S_2 S_3]$~~

~~S_4~~

~~$[S_5 S_6 S_7 S_8]$~~

	0	1
S_1	S_1	S_2
S_2	S_1	S_3
S_3	S_4	S_3
S_4	S_1	S_5
*$[S_5 S_6 S_7]$	$[S_5 S_6 S_7 S_8]$	$[S_5 S_6 S_7 S_8]$

	a	b
A	BF	BF
BF	CDFG	CDFG
*CDFG	HI	HI
HI	HI	HI

	a	b
→ A	B	F
B	A	F
C	G	A
D	H	B
E	A	G
* F	H	C
* G	A	D
* H	A	C

Q.)

B
C x x
D x x o
E x x x x
F x x x x x
G x x x x x x
H x x x x x x o
A B C D E F G

① {F, G, H}

{A, B, C, D, E}

(F, G) on a → (H, A)

(F, H) on a → (H, A)

(G, H) ✗ similar

(A, B), (A, E) (B, E) (C, D)
~~(B, H)~~ (B, H)

(A, C) on a → (B, G)

(A, D) on a → (B, H)

~~(B, F)~~

(B, C) on a → (A, G)

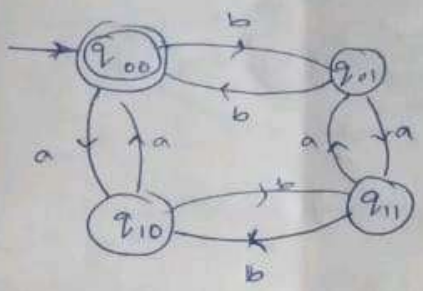
(B, D) on a → (A, H)

(C, E) on a → (G, A)

(D, E) on a → (H, A)

If in DFA, (NFA-to DFA) conversion we get empty state then we have to take an extra state (dead state)

18) construct a finite automata over {a,b} no. of a's & b's divisible by 2.



Give meaning to the state

20) no. of 'a's divisible by 2 & no. of b's divisible by 3

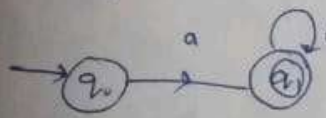
a → 2 b → 3

2 × 3 = 6

* If we can build automata then it is Regular Language.

→ $L_1 = \{a^n \mid n \geq 1\}$

n repetitions of 'a'



→ if $\{a^n \mid n \geq 0\}$ → 

→ Any no.

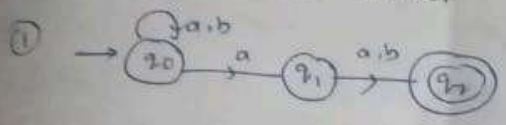
$a^* = \{\epsilon, a, aa, aaa \ldots\}$

$a^1 = \{a\}$

$a^2 = \{aa\}$

NFA to DFA conversion

* NFA must be correct

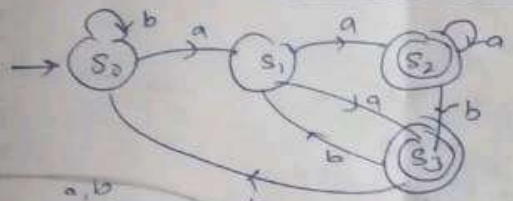
① 

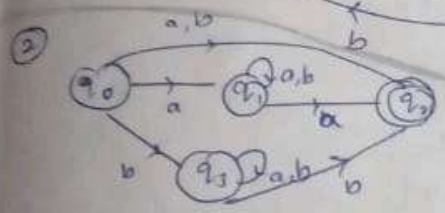
NFA

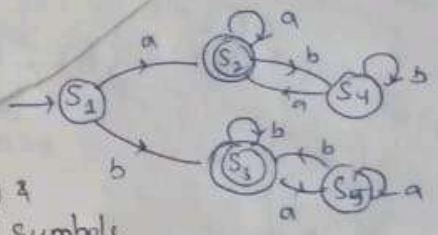
	a	b
q_0	q_0q_1	q_0
q_1	q_2	q_2
*q_2	–	–

DFA

		a	b
S_2	q_0	$[q_0q_1]$	q_0
S_2	$[q_0q_1]$	$[q_0q_1q_2]$	$[q_0q_2]$
*S_3	$[q_0q_1q_2]$	$[q_0q_1q_2]$	$[q_0q_2]$
*S_4	$[q_0q_2]$	$[q_0q_1]$	q_0



② 



starting & ending symbols all same

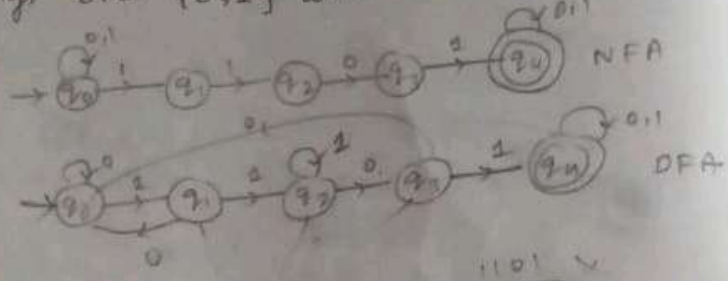
NFA

	a	b
q_0	q_1q_2	q_2q_3
q_1	q_1q_2	q_1
q_3	q_3	q_2q_3
*q_2	–	–

DFA

		a	b
S_1	q_0	$[q_1q_2]$	$[q_2q_3]$
*S_2	$[q_1q_2]$	$[q_1q_2]$	q_1
*S_3	$[q_2q_3]$	$[q_3]$	$[q_2q_3]$
S_4	q_1	$[q_1q_2]$	q_1
S_5	q_3	q_3	$[q_2q_3]$

construct a finite automata that accepts binary strings over {0,1} which contain 1101 substring



1101 ✓

01101 0

000

$\delta(q_0, 0)$

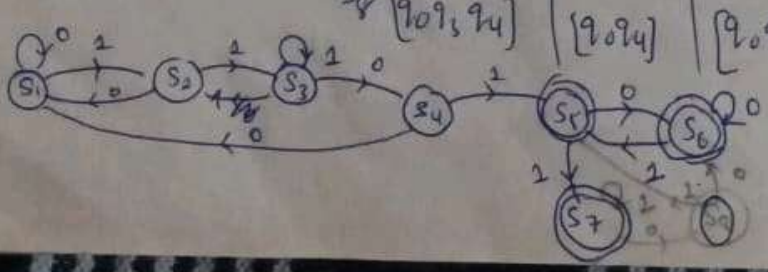
$\delta(q_0, 1)$. 12 $\xrightarrow{1}$ q_0 01 ✓

$\delta(q_3, 0)$ 110 0 1101 ✗

dead state x once we're in deadstate then we'll stay the

	0	1
q_0	q_0	q_0q_1
q_1	–	q_2
q_2	q_3	–
q_3	–	q_4
$*q_4$	q_4	q_4

		0	1
S_1	q_0	q_0	$[q_0q_1]$
S_2	$[q_0q_1]$	q_0	$[q_0q_1q_2]$
S_3	$[q_0q_1q_2]$	$[q_0q_3]$	$[q_0q_1q_2]$
S_4	$[q_0q_3]$	q_0	$[q_0q_1q_4]$
$*S_5$	$[q_0q_1q_4]$	$[q_0q_4]$	$[q_0q_1q_2q_4]$
$*S_6$	$[q_0q_4]$	$[q_0q_4]$	$[q_0q_1q_4]$
$*S_7$	$(q_0q_1q_2q_4]$	$[q_0q_3q_4]$	$[q_0q_1q_2q_4]$
$*S_8$	$[q_0q_3q_4]$	$[q_0q_4]$	$[q_0q_1q_4]$



NFA

* NFA

①

NFA

q_0	
q_1	
$*q_2$	

②

NFA

q_0
q_1
q_3
$*q_2$

Deterministic f A:

From a state, On an i/p symbol, if there exists exactly one transition then it is said to be deterministic finite Automata

$\delta : Q \times \Sigma \longrightarrow Q$

δ : Transition function. $Q \times \Sigma$: Domain. Q : Range

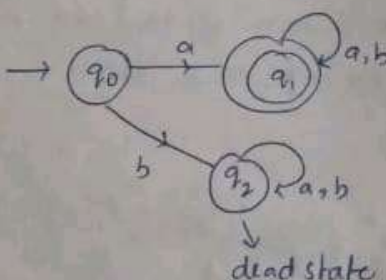
Non-Deterministic f-A:

From a state, on an i/p symbol, if there may exist 0 or any no. of transitions

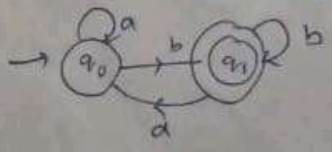
Transition

$\delta : Q \times \Sigma \rightarrow 2^Q$ (Power set of Q. all possibilities)

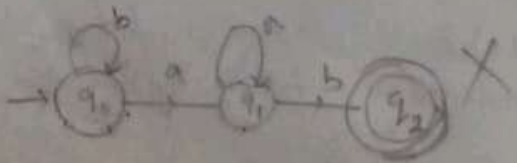
2Q)



3Q)



4Q) no of



finite automata

FA = $(Q, \Sigma, \delta, q_0, F)$

Q: set of finite no of states

Σ: Input alphabet

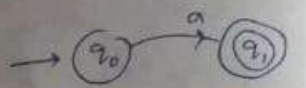
δ: transition function

q_0: Initial state

F: set of final states

next state combination

for each edge, we have transition



Q $\{q_0, q_1\}$

Σ: $\{a\}$

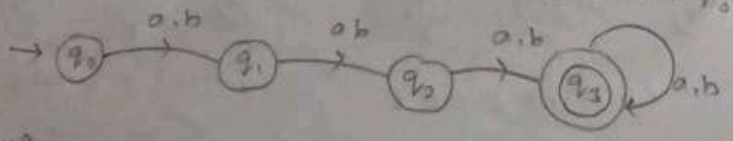
δ: $\delta(q_0, a) = q_1$

q_0: q_0

F: $\{q_1\}$

1Q) Construct a finite automata that accepts all the strings over $\{a, b\}$ whose length is atleast 5. With how many states, you can build the automata

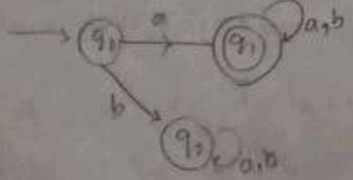
→ always see existing states can be used or not

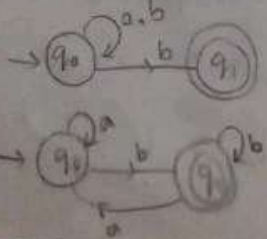


DFA

$L = \{aaaa, aaab, abaa$

Here we need not to go back to the starting state, because after 3, we can accept all the other strings





Subsequence Picking any letters from strings

no of subsequence - 2^n (no of subsets)

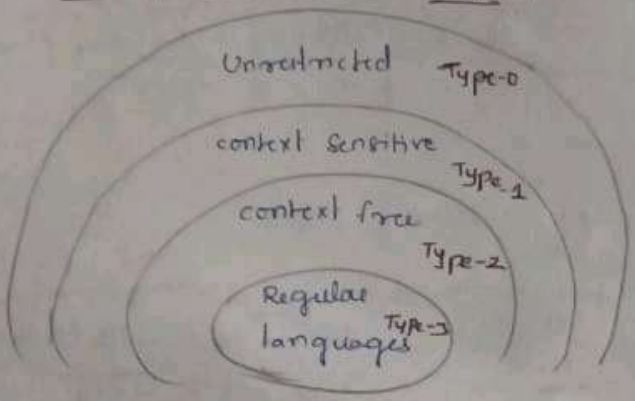
$= {}^nC_0 + {}^nC_1 + \dots + {}^nC_n$

ϵ → empty string

ϕ → empty set

* A lang. can have 1 string ∅ that can be 'ε'

→ Chomsky Hierarchy of languages-

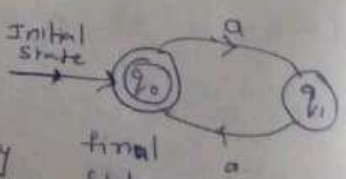


* Finite Automata / finite State Machine:-

can be build by finite States | The automata with finite no. of states

After reading a symbol, it is mean.

Why finite state?



Starting from initial state, by reading what you will reach final state forms the lang.

$L = \{\epsilon, a, aa, aaaa, \dots\}$

Automata Theory:- (It is mathematical study of machine that read & process strings)

1. Alphabet: Set of Symbols over which the language is formed ' Σ '
2. String: Sequence of Symbols (order & combination) (w)
3. Language: set of string formed over Alphabets ' L '

Prefixes: no of prefixes in a word 'prefix' (n+1)

Prefix
Prefi
Pref
Pre
Pr
P
ϵ → empty (without any Symbols)

" It is a substring of a string formed by removing an end substring ".

Suffixes: Suffix
uffix
ffix
fix
ix
x
ϵ

Substring:-
Remove the symbols from either start/end of string then the remaining:
no. of Substring: $\frac{n(n+1)}{2} + 1$

Bcz: " substring "

1 length: s u b s t r i n g
2 length: su ub bs st tr ri in ng

r length: substring.

Empty: ϵ

$\frac{3(3+1)}{2} = \frac{3 \times 4}{2} = 6$

3(3)